

Journal Pre-proof

A new branch-and-cut approach for integrated planning in additive manufacturing

Benedikt Zipfel, Felix Tamke, Leopold Kuttner

PII: S0377-2217(24)00843-9
DOI: <https://doi.org/10.1016/j.ejor.2024.10.040>
Reference: EOR 19252

To appear in: *European Journal of Operational Research*

Received date: 5 April 2023

Accepted date: 28 October 2024



Please cite this article as: B. Zipfel, F. Tamke and L. Kuttner, A new branch-and-cut approach for integrated planning in additive manufacturing. *European Journal of Operational Research* (2024), doi: <https://doi.org/10.1016/j.ejor.2024.10.040>.

This is a PDF file of an article that has undergone enhancements after acceptance, such as the addition of a cover page and metadata, and formatting for readability, but it is not yet the definitive version of record. This version will undergo additional copyediting, typesetting and review before it is published in its final form, but we are providing this version to give early visibility of the article. Please note that, during the production process, errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

© 2024 Published by Elsevier B.V.

A new branch-and-cut approach for integrated planning in additive manufacturing

Benedikt Zipfel^{a,*}, Felix Tamke^a and Leopold Kuttner^a

^aTU Dresden, Department of Business Administration, esp. Industrial Management, Helmholtzstraße 10, Dresden, 01069, Saxony, Germany

ARTICLE INFO

Keywords:

Scheduling

Unrelated machines

Orthogonal packing

Additive manufacturing

Branch-and-cut

ABSTRACT

In recent years, there has been considerable interest in the transformative potential of additive manufacturing (AM) since it allows for producing highly customizable and complex components while reducing lead times and costs. The rise of AM for traditional and new business models enforces the need for efficient planning procedures for AM facilities. In this area, the assignment and sequencing of components to be built by an AM machine, also called a 3D printer, is a complex challenge combining two combinatorial problems: The first decision involves the grouping of parts into production batches, akin to the well-known bin packing problem. Subsequently, the second problem pertains to the scheduling of these batches onto the available machines, which corresponds to a parallel machine scheduling problem. For minimizing makespan, this paper proposes a new branch-and-cut algorithm for integrated planning for unrelated parallel machines. The algorithm is based on combinatorial Benders decomposition: The scheduling problem is considered in the master problem, while the feasibility of an obtained solution with respect to the packing problem is checked in the sub-problem. Current state-of-the-art techniques are extended to solve the orthogonal packing with rotation and used to speed up the solution of the sub-problem. Extensive computational tests on existing and new benchmark instances show the algorithm's superior performance, improving the makespan by 18.7% on average, with improvements reaching up to 97.6% for large problems compared to an existing integrated mixed-integer programming model.

1. Introduction

Originally, additive manufacturing (AM) was adopted to accelerate time-to-market and quickly identify solutions through rapid prototyping during the pre-production phase. However, it is now increasingly being utilized for serial production across various industries, including aerospace, automotive, and medical sectors (Kang et al., 2018). The aerospace industry, in particular, benefits from implementing AM technologies, notably in reducing the weight of components. This weight reduction contributes to lower energy consumption and decreased emissions, aligning with environmental sustainability goals (Blakey-Milner et al., 2021). For instance, GE Aerospace utilizes AM to manufacture fuel nozzle tips for their engines, resulting in a 25% reduction in weight and streamlining 20 individual components to a single integrated part (GE Aerospace, 2018).

The general technology, also known as 3D printing, can be described as the process of building up parts layer by layer, based on a digital data model (Thompson et al., 2016). AM technologies do not require prior tooling activities (Attaran, 2017) and are able to process several parts simultaneously in the same production step (Kucukkoc, 2019). Because AM enables the building of highly complex parts, it is especially suited to customization purposes (Guo and Leu, 2013). This customization is particularly valuable in the medical industry, where patient-specific implants and prosthetics can be precisely tailored to individual anatomy. While research initially studied the technologies and processes, recent studies

*Corresponding author

 benedikt.zipfel@tu-dresden.de (B. Zipfel); felix.tamke@tu-dresden.de (F. Tamke); leopold.kuttner@tu-dresden.de (L. Kuttner)

ORCID(s): 0000-0003-3750-2541 (B. Zipfel); 0000-0003-1650-8936 (F. Tamke); 0000-0002-9359-8835 (L. Kuttner)

increasingly focus on production planning aspects of AM (Oh et al., 2020). The growing interest in production planning is partly driven by new on-demand services, such as Factory-as-a-Service and Production-as-a-Service, that are based on AM (see, e. g., Kang et al. (2018)). However, efficient planning and operations management are also essential for traditional industries shifting from prototypical usage of AM technologies to application in series production.

This study addresses the integrated planning problem on AM machines, which incorporates the nesting of parts into batches and the scheduling of those batches on the available machines (Manco et al., 2019). Since the nesting and scheduling problems are closely interdependent (Oh et al., 2020), addressing both sub-problems simultaneously is essential for achieving high-quality schedules (Kapadia et al., 2019). Efficient nesting is particularly important because maximizing space utilization reduces the number of batches requiring scheduling, which in turn minimizes the number of machine setups. Additionally, in some AM technologies, the build height of a batch affects the processing time, meaning that denser packings can have a significant impact on overall production times. The necessity of an integrated solution procedure becomes even more apparent in considerations of unrelated parallel machines with different build spaces and processing speeds. Therefore, we propose a new integrated exact solution approach to minimize makespan in an AM environment with unrelated parallel machines. The main contributions of this paper are the following:

- We present a novel branch-and-cut (B&C) solution procedure for minimizing the makespan on unrelated parallel AM machines, leveraging combinatorial Benders decomposition.
- We extend several state-of-the-art techniques from the literature—such as lower bounding, relaxation, placement point strategies, and constraint programming models—to address the sub-problem of two-dimensional orthogonal packing with rotation.
- We compare our approach with another exact approach from the literature, and demonstrate the superior performance of our algorithm by providing more optimally solved instances in less computation time.
- We propose new benchmark instances for the considered planning problem and assess the impact of part sizes on the resulting schedules and objectives, providing valuable insights for decision-makers.
- We demonstrate the effectiveness of the proposed algorithmic components based on comprehensive analyzes and evaluate alternative solution strategies for decision-making in the packing sub-problem.

In this study, we focus on powder bed fusion (PBF) techniques, which allow for the processing of a wide range of materials, including metals, ceramics, and polymers. As a result, PBF technologies are used across various industries such as aerospace, maritime, and automotive. Additionally, PBF is also utilized in the production of medical devices (Fina et al., 2018). The overall time for part construction is longer in PBF compared to other AM techniques, primarily due to the necessary preheat and cool-down cycles. However, advancements in machine design, such as removable build platforms, have enabled these cycles to take place off-line; this innovation enhances machine productivity, allowing for efficient use of PBF technology (Gibson et al., 2021). In PBF technologies, a leveling roller spreads powder on the build area, after which a laser beam melts designated areas of the powder to create a layer of the part. The build platform is lowered, and the process is repeated until the parts are printed completely (Gibson et al., 2021). Consequently, the processing time depends on the maximum height and the total volume that needs to be printed.

The considered problem involves two combinatorial problems. The nesting problem typically focuses on the optimal placement and orientation of parts within the build area. If there is insufficient space to accommodate all required parts into a single build area, the problem extends to include the decision of grouping parts into several batches (Oh et al., 2020). In this study, we consider the bounding boxes of each part in the nesting problem. This is illustrated in Figure 1, where the cuboid bounding box of a model part is represented by dashed lines. The terms 'nesting' and 'packing' are sometimes used as synonyms; however, as we understand it, nesting is associated with irregular shapes (see, e. g., Baldacci et al., 2014), whereas packing is mostly associated with rectangular shapes. Given that this study considers cuboid bounding boxes with rectangular base areas, we therefore use the term 'packing' in the remainder of this work. In addition, since we do not allow the stacking of parts, each part must be connected to the build platform of a printer. Furthermore, we assume a fixed build orientation of each part, which is predefined in the design process. Parts can only be rotated 90 degrees around the z-axis, as illustrated in Figure 1, which shows both possible rotations of an exemplary part. Rotating a part maintains its height while interchanging its width and length.

Branch-and-cut for integrated planning in additive manufacturing

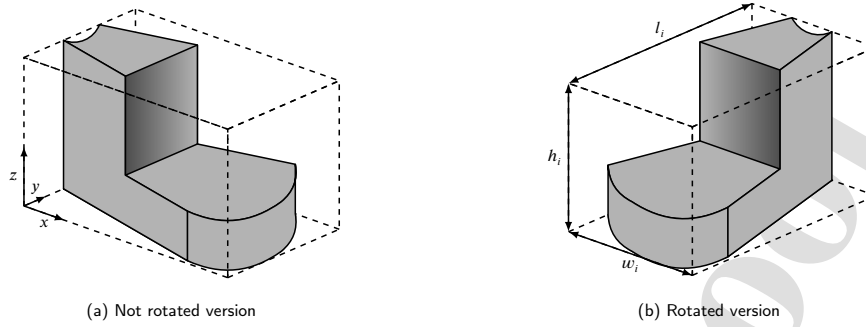


Figure 1: Illustration of the considered bounding boxes of a part

From a scheduling perspective, the considered problem is similar to traditional batch processing scheduling problems (BPSPs), where batch processors are able to produce several jobs simultaneously within a single production step. Consequently, the BPSP comprises two decisions: assigning jobs to batches and determining the sequence of those batches on the available machines. In comparison to the BPSP, we see two major differences in planning for AM. First, in traditional batch scheduling, the processing time of a batch is typically defined by the maximum or the sum of all job processing times, while the processing time in AM depends on the composition of a batch (Alicastro et al., 2021). This contrast is further exemplified by recalling the build process of PBF technologies discussed earlier. Second, in conventional batch scheduling, the batching of jobs is mostly done using a batch size limit and a one-dimensional packing constraint with identical or non-identical job sizes (Fowler and Mönch, 2022). In AM, the packing is more relevant to ensure a feasible and error-free printing process. In summary, the studied problem incorporates both the packing of parts as a two-dimensional bin packing problem with rotation (2D-BPR) and the scheduling of the resulting batches on the available unrelated machines, which differ in terms of their processing speeds and build spaces.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature in scheduling problems, packing problems, and planning for AM. In Section 3, we define the studied problem and formulate a mixed-integer linear programming model. Section 4 presents the structure of the proposed solution approach in detail; we give an overview of the general solution strategy before elaborating on the specific elements of the approach. In our computational studies in Section 5, we introduce and analyze the used data before presenting comprehensive test results. Finally, we conclude in Section 6 and offer suggestions for future research.

2. Related work

We review key articles from literature streams that are most relevant to the problem configuration considered here. Given the problem's similarities to the BPSP, we begin by examining studies in this area, followed by a review of pertinent research from the packing and cutting literature. Lastly, we highlight recent papers that specifically address packing and scheduling for AM.

2.1. Scheduling on batch processing machines

Scheduling on batch processing machines has been studied extensively in recent decades. Initially, Uzsoy (1994) introduced the BPSP for a single batch processing machine; since then, several authors have extended the problem. The most relevant extensions related to the problem examined in this study are non-identical job sizes and unrelated parallel machines. Li et al. (2013) study batch scheduling with these two extensions to minimize makespan, combining different heuristics in two-step approaches and evaluating their performance. More recently, Arroyo and Leung (2017b) propose a mathematical programming model for a similar problem, extended by unequal ready times; along with the exact solution approach, they propose several heuristics to solve the problem. Similar to the present study, both of these studies take into account parts with different sizes; however, the machines in those studies differ solely in processing times, while in our case the machines also differ in capacity. In Arroyo and Leung (2017a), the authors extend their previous work and additionally address machine-dependent processing times of the parts and different machine capacities. Muter (2020) develop an exact algorithm for the single batch processing machine scheduling

problem to minimize makespan. The proposed column-and-cut algorithm incorporates a column generation approach, utilizing a labeling algorithm to address the pricing subproblems effectively. Emde et al. (2020) aim to minimize maximum lateness in a single machine batch processor problem. The authors propose a solution approach based on logic-based Benders decomposition that outperforms best-known solution approaches from the literature.

In contrast to the present problem, which assumes a two-dimensional packing of parts, all previous articles consider a one-dimensional packing case. A combination of batch scheduling and two-dimensional bin packing is found in Bennell et al. (2013), where the authors consider the problem of a single batch processing machine with a constant processing time regardless of the packed parts. The developed genetic algorithms aim to minimize the number of batches and the maximum lateness. In Polyakovskiy and M'Hallah (2018) and Polyakovskiy and M'Hallah (2021), the authors consider similar problems with non-guillotine and guillotine packing restrictions, respectively; however, in these studies, the problems are solved by introducing different combinations of mixed integer programming (MIP) models and constraint programming (CP) models. In addition to the problem configurations and objectives, our problem configuration differs from the one in Polyakovskiy and M'Hallah (2021) by considering non-guillotine packing, where rectangles do not necessarily need to be placed at bisecting edge-to-edge lines across the bin, while they need to do for guillotine packing (see, e. g., Amossen and Pisinger, 2010). For a comprehensive overview of recent literature on batch scheduling problems, interested readers are referred to the detailed analysis by Fowler and Mönnch (2022).

2.2. Two-dimensional packing

Iori et al. (2021) provide a comprehensive overview of exact solution approaches for different problems related to two-dimensional packing. In this study, the packing problem can be formally defined as a two-dimensional bin packing problem (2D-BPP) with two extensions: i) variable bin sizes and ii) orthogonal rotations. In addition, when considering each batch of parts individually, the packing problem can be described as a two-dimensional orthogonal packing problem with rotation (2D-OPR), which is, in turn, a variant of the two-dimensional orthogonal packing problem (2D-OPP). Therefore, we review relevant articles with those problem configurations.

Arbib et al. (2021) address the 2D-BPP with multiple objectives using heuristic procedures. In contrast, Côté et al. (2021) focus on exact algorithms and propose a branch-and-cut approach based on Benders decomposition for minimizing the number of bins; they incorporate several state-of-the-art mechanisms and methods from the packing literature to improve the best-known results for the benchmark sets. While Côté et al. (2021) consider the 2D-BPP, in our work, the packing is a sub-problem of the integrated problem configuration. Nevertheless, the ideas presented in that study significantly influenced the investigations in our paper and will thus be referred to in different sections of this study. Pisinger and Sigurd (2005) investigate the 2D-BPP with variable bin sizes. The authors propose an exact solution approach based on branch-and-price, and also present several lower bounds for the problem, including a bound based on Dantzig-Wolfe decomposition. Liu et al. (2011) develop two heuristics based on dynamic programming for the same problem. To handle the large number of packing states, the authors aggregate packing states with similar characteristics. A goal-driven heuristic is proposed by Wei et al. (2013) for the 2D-BPP with variable bin sizes, which is later adapted by Gardeyn and Wauters (2022) to address an extended problem incorporating orthogonal rotation and guillotine cuts. The authors develop a heuristic that iteratively destroys and reconstructs parts of the solution, while successively reducing the total bin area limit.

Several other publications in the packing literature discuss orthogonal rotation of parts, whereby we first focus on lower bounding techniques. Dell'Amico et al. (2002) present a lower bound for 2D-BPP with rotation (2D-BPR), for which the parts are cut into squares; the new lower bounding technique is embedded in a branch-and-bound algorithm, and its efficiency against the continuous lower bound is demonstrated in a computational study. Boschetti and Mingozzi (2003) propose another lower bound by modifying part sizes. A summary of lower bounds for the 2D-BPR is presented by Clautiaux et al. (2007b), who also present a new lower bound for the 2D-BPR if the considered bin is a square; they demonstrate the dominance of the newly proposed bound in computational tests. Polyakovskiy and M'Hallah (2018) adapt the developments on dual feasible functions (DFFs) proposed in Fekete and Schepers (2004), and present yet another bound for the 2D-BPR; they show the superior performance of their bound compared to the lower bound of Clautiaux et al. (2007b) and the bound resulting from CPLEX.

The 2D-OPP and 2D-OPR are primarily considered when solving the two-dimensional strip packing problem (2D-SPP) by first determining a lower bound on the strip height, then looking for a feasible solution to the respective height. The literature on these problems is rich, and the problems are studied using several methods. Martello et al. (2003) propose

the bin packing problem with contiguity constraints (1CBP) as a relaxation of the 2D-SPP; The 1CBP is later adopted for the 2D-OPP by Belov et al. (2009), who present a new relaxation for which the continuity of the part strips is not necessary; this relaxation is often referred to as the 'bar relaxation' in the literature (see, e.g., Belov et al., 2013). Other B&B algorithms are presented by Clautiaux et al. (2007b), Kenmochi et al. (2009), and Fleszar (2016). While the procedure of Clautiaux et al. (2007b) considers the 2D-OPP without orthogonal rotation of parts, the other approaches account for both fixed part orientation and rotatable parts. Kenmochi et al. (2009) propose a B&B algorithm for the two-dimensional perfect packing problem with non-guillotine packing patterns. In addition, the latter develop different techniques to extend the algorithm for solving the 2D-SPP with and without part rotation; the resulting algorithms iteratively check for a feasible packing of parts given a fixed height. Côté et al. (2014) introduce another solution technique for the 2D-SPP based on Benders decomposition: they propose a parallel-machine scheduling problem with contiguity restrictions as their master problem, while the sub-problem checks for a feasible packing of the solution candidate. The 2D-SPP with orthogonal rotation is solved in Delorme et al. (2017) with a similar approach. First, the authors try to find a feasible x -coordinate for each part using an arc-flow formulation of the 1CBP. Thereafter, the slave problem checks whether a feasible alignment along the y -coordinate exists for a given solution candidate for the master problem.

Throughout the packing literature, several authors propose rules to reduce the complexity of packing problems; these rules involve placement patterns as well as problem reductions. Herz (1972) introduces the principle of normal patterns, to reduce the number of possible placement points for each part without losing optimality. More recently, Côté and Iori (2018) propose another placement scheme, denoted as meet-in-the-middle patterns, which further reduces the number of potential positions. Boschetti and Montaletti (2010), who study the 2D-SPP, summarize several techniques to reduce the complexity of the problem by changing the dimensions of items and bins. These methods involve increasing part sizes (Boschetti et al., 2002; Carlier et al., 2007), reducing bin sizes (Alvarez-Valdés et al., 2009), and fixing some parts at the bottom of the strip (Martello et al., 2003; Carlier et al., 2007).

2.3. Integrated planning for additive manufacturing

2.3.1. Integrated planning with one-dimensional packing

The number of studies explicitly incorporating packing and scheduling for AM is growing very quickly. Li et al. (2017) introduce a MIP model to minimize production costs for the problem of scheduling AM machines; they add tolerances to the production areas of the parts, and they check whether the summed area of assigned parts is smaller than the available build area on the machine. This one-dimensional bin packing problem (1D-BPP) constraint is adopted by Kucukkoc (2019), who studies scheduling in AM for single, identical, and unrelated parallel machines: for each machine environment, the author presents an MIP model that aims to minimize makespan. Tavakkoli-Moghaddam et al. (2020) tackle makespan and total tardiness simultaneously in a bi-objective approach; in addition, they extend the work of Kucukkoc (2019) by considering material types, which leads to incompatibilities between products. Zipfel et al. (2021) link the extensions of Tavakkoli-Moghaddam et al. (2020) with the order-based configuration of Chergui et al. (2018) and propose a new MIP formulation. In addition to different material types, the authors also incorporate quality levels for the ordered parts. Elsewhere, a matheuristic for minimizing total weighted tardiness is proposed by Rohaninejad et al. (2021), who present a combination of a genetic algorithm and a local search based on an MIP model: the machine assignment is addressed in the genetic algorithm, while the MIP-based local search takes care of assigning parts to batches. In Alicastro et al. (2021), an iterated local search is combined with reinforcement learning; the authors further adopt heuristic approaches from strip packing problems to place the parts within the available batches.

2.3.2. Integrated planning with two-dimensional packing

All above-mentioned approaches for integrated planning in AM share in common that they use a 1D-BPP constraint instead of explicitly solving the packing problem. This can be a valid assumption if parts are large compared to the build area. However, to meet real-world requirements, the specific dimensions of the parts must be taken into account. Che et al. (2021) introduce an MIP model that incorporates the scheduling problem and a two-dimensional packing problem that considers the placement and the orientation of cuboid parts with a rectangular base area on the build area. The orientation variants of each part are predefined by selected rotation movements around the x - and y -axes, while rotations around the z -axis are considered in the model formulation. The model handles orientations around the x - and y -axes using a constraint that allows only one orientation variant of each part to be placed. In addition to the mathematical model, Che et al. (2021) present a simulated annealing procedure, which they combine with different packing strategies and post-optimization methods; in a comprehensive computational test, they show the efficiency

of their developed algorithms. In Hu et al. (2022), the aforementioned problem is extended by unequal release dates of parts; the authors adapt the MIP model presented in Che et al. (2021) and incorporate constraints to account for the extended problem configuration. Furthermore, they use an adaptive large neighborhood search to solve large-scale instances of the problem. Yu et al. (2022) consider the problem of minimizing tardiness and propose a mathematical formulation; in addition to possible rotations of parts, they integrate the determination of build orientation within their problem structure. In Lee and Kim (2023), the authors also consider all rotation variants of parts with the objective of minimizing makespan, and develop population-based metaheuristics based on the frameworks of genetic algorithms and particle swarm optimization. Liu et al. (2023) present an enhanced exact solution procedure based on branch-and-price. In contrast to our configuration, the authors consider a single-machine environment and two-dimensional packing. They evaluate the algorithm using industry data and compare the results with an integrated model formulation. Very recently, Mao et al. (2024) study the single-machine scheduling problem with two-dimensional packing in AM environments. To solve the problem, the authors use a Benders decomposition approach similar to the one proposed in this work. The conducted computational studies reveal a significant performance improvement compared to a MIP formulation. Moreover, the algorithm yields high quality results compared to heuristics. In contrast to the work in Mao et al. (2024), this study considers rotation of parts, and the presence of unrelated parallel machines.

Only few studies consider irregular-shaped parts instead of rectangular ones. Zhang et al. (2020) present a mathematical formulation and propose a population-based metaheuristic that integrates the principles of no-fit polygons and inner-fit polygons. These techniques are also used by Nascimento et al. (2023), who introduce a CP model for integrated packing and scheduling of irregular parts. In a follow-up paper, Nascimento et al. (2024) present logic-based Benders decomposition procedures similar to the methodology in this work. The authors propose variants of the algorithm using a MIP and a CP formulation for the master problem. The decomposition approaches yield significantly better results compared to an integrated model formulation.

Very recently, Pinto et al. (2024) review the current literature on packing and scheduling problems in AM. The survey shows that most of the studies on integrated scheduling AM with two-dimensional rectangular packing focus on developing heuristic procedures, while literature on enhanced exact approaches is scarce. Apart from mathematical formulations, only the study by Liu et al. (2023) and Mao et al. (2024) present a sophisticated exact approaches. However, they both consider single-machine environments. The lack of efficient exact approaches for planning in AM emphasizes the importance of the present work.

In summary, this study examines minimizing makespan on unrelated batch processing machines in AM, where the batching of parts is represented by a 2D-BPR and variably-sized bins. This problem configuration is an extension of the planning situation seen in Kucukkoc (2019) as well as a special case of the problem in Che et al. (2021) and Hu et al. (2022). The studied scheduling sub-problem is a generalization of single batch processing machine scheduling, which has been proven to be NP-hard for minimizing makespan (Uzsoy, 1994). Furthermore, the packing sub-problem can be reduced to the 1D-BPP; therefore, it is also strongly NP-hard. In contrast to most existing work on integrated planning for AM, in this work, we explicitly focus on creating a comprehensive exact approach. Conceptually, the procedures build on and extend the methodologies in the literature of packing problems presented in Section 2.2. The following section gives a description of the problem configuration, followed by a mathematical formulation.

3. Problem description

3.1. Assumptions

For the studied problem, we make the following assumptions:

- All information regarding parts and machines is known and deterministic.
- We consider 3D printers with PBF technology as unrelated parallel machines that may vary in terms of their sizes and production parameters.
- All printers are available from the beginning of the planning period and there are no breakdowns of production or malfunctions.
- We consider each part using its bounding box and the respective dimensions (see Figure 1).
- Each part has a predefined build orientation and is only allowed to rotate 90 degrees around the z-axis.

Branch-and-cut for integrated planning in additive manufacturing

- The volume of a part can be smaller than the cuboid volume of the bounding box of its width, length, and height.
- The machines are able to build several parts simultaneously in batches, and we do not allow the stacking of parts within the build spaces.
- A part needs to be assigned to exactly one batch on one specific machine.
- As soon as the production of a batch starts, no part can be added to or removed from the batch.
- The maximum number of batches is equal to the total number of parts on each machine, to guarantee feasibility.

By considering the bounding boxes of the parts, we aim to ensure consistent part quality by reducing interference of parts with each other. Furthermore, the build orientation of a part has a significant impact on the later quality of the produced component. Therefore, orientation in the build space is often determined in an earlier process step (Kucukkoc, 2019). Different materials can be used in PBF technology. Depending on the material, additional support structures are needed to ensure the accurate printing of complex structures such as hollow spaces and overhangs. The stacking of parts depends on the aforementioned choice of material and need for support structures (Bain, 2019). In this study, we assume that support structures are required to stabilize overhangs and prevent deformation during printing. Thus, stacking parts within the build space is not permitted. This assumption is important, as it reduces the packing subproblem from three-dimensional and a two-dimensional packing. However, even with a two-dimensional packing approach, we must ensure that no part exceeds the build space in terms of height.

3.2. Problem definition

Let $\mathcal{I} = \{1, 2, \dots, n\}$ be a set of parts, each of which is defined by its width w_i , length l_i , and height h_i , $i \in \mathcal{I}$. Additionally, we are given a set $\mathcal{M} = \{1, 2, \dots, m\}$ of machines, which use PBF technology. The build envelope of an AM machine $k \in \mathcal{M}$ is defined by its width W_k , length L_k , and height H_k . Besides their variegated build spaces, 3D printers also differ in terms of production times, which can be divided into setup time S_k^S , recoating time S_k^R , and scanning time S_k^L . The recoating time represents the time to apply a height unit of powder onto the build area, while the scanning time refers to the time the laser needs to build one volume unit (Kucukkoc, 2019). Consequently, the processing time of a batch comprises the recoating time for the highest part and the scanning time for the total volume within the respective batch. The left side of Figure 2 illustrates a model instance of the studied problem with ten parts and two machines, and the right side of the figure presents a possible solution for this. Three batches have been created from the ten parts, based on the available build space of the machines, and those batches are sequenced on the two machines. It should be noted that we numbered the batches in ascending order for each machine – for example, Batch_{1,1} is the first batch on the first machine.

In line with the example in Figure 2, the studied problem consists of assigning each part $i \in \mathcal{I}$ to exactly one machine $k \in \mathcal{M}$, packing those parts into a set of batches $\mathcal{B} = \{1, 2, \dots, b\}$, placing them within the respective build spaces, and sequencing the resulting batches on the machines. The objective is to minimize the makespan C_{max} in a machine environment of unrelated parallel machines.

3.3. Problem formulation

To describe the studied problem mathematically, we modify the formulation of Kucukkoc (2019) by incorporating the ideas of Côté et al. (2021) for the 2D-BPP. Before modeling the problem, we give a complete description of the used notation:

Classes & Sets			
$\mathcal{B}$	Set of batches	$\mathcal{I}$	Set of parts
$\mathcal{M}$	Set of machines	$\mathcal{S}$	Generic subset of parts, $\mathcal{S} \subseteq \mathcal{I}$
$\mathcal{P}_k$	Class of infeasible subsets for part assignments to machine k	$\mathcal{S}_k^{IA}$	Subset of parts $i \in \mathcal{I}$, where part i cannot be placed on $k \in \mathcal{M}$
Parameters			
a_i	Production area of part $i \in \mathcal{I}$	h_i	Height of part $i \in \mathcal{I}$
l_i	Length of part $i \in \mathcal{I}$	v_i	Volume of part $i \in \mathcal{I}$
w_i	Width of part $i \in \mathcal{I}$	A_k	Area of machine $k \in \mathcal{M}$

Branch-and-cut for integrated planning in additive manufacturing

H_k	Height of machine $k \in \mathcal{M}$	L_k	Length of machine $k \in \mathcal{M}$
W_k	Width of machine $k \in \mathcal{M}$	S_k^L	Scanning time for one volume unit on machine $k \in \mathcal{M}$
S_k^R	Recoating time for one height unit on machine $k \in \mathcal{M}$	S_k^S	Setup time on machine $k \in \mathcal{M}$
Decision Variables			
$\hat{c}_{jk}$	Completion time of batch $j \in \mathcal{B}$ on machine $k \in \mathcal{M}$	$\hat{h}_{jk}$	Height of batch $j \in \mathcal{B}$ on machine $k \in \mathcal{M}$
$\hat{x}_{ijk}$	1, if part $i \in \mathcal{I}$ is assigned to batch $j \in \mathcal{B}$ on machine $k \in \mathcal{M}$	$\hat{y}_{jk}$	1, if batch $j \in \mathcal{B}$ is processed on machine $k \in \mathcal{M}$
C_{max}	Makespan		

With this notation on hand, the studied problem can be stated as an MIP model as follows:

$$C_{max} \rightarrow \min \quad (1)$$

s.t.:

$$C_{max} \geq \hat{c}_{jk} \quad \forall j \in \mathcal{B}, k \in \mathcal{M}, \quad (2)$$

$$\sum_{j \in \mathcal{B}} \sum_{k \in \mathcal{M}} \hat{x}_{ijk} = 1 \quad \forall i \in \mathcal{I}, \quad (3)$$

$$\hat{x}_{ijk} \leq \hat{y}_{jk} \quad \forall i \in \mathcal{I}, j \in \mathcal{B}, k \in \mathcal{M}, \quad (4)$$

$$\hat{h}_{jk} \geq h_i \cdot \hat{x}_{ijk} \quad \forall i \in \mathcal{I}, j \in \mathcal{B}, k \in \mathcal{M}, \quad (5)$$

$$\hat{x}_{ijk} = 0 \quad \forall j \in \mathcal{B}, k \in \mathcal{M}, i \in S_k^{IA}, \quad (6)$$

$$\hat{c}_{1,k} \geq \hat{y}_{1,k} \cdot S_k^S + S_k^L \cdot \sum_{i \in \mathcal{I}} (v_i \cdot \hat{x}_{i,1,k}) + S_k^R \cdot \hat{h}_{1,k} \quad \forall k \in \mathcal{M}, \quad (7)$$

$$\hat{c}_{jk} \geq \hat{c}_{j-1,k} + \hat{y}_{jk} \cdot S_k^S + S_k^L \cdot \sum_{i \in \mathcal{I}} (v_i \cdot \hat{x}_{i,j,k}) + S_k^R \cdot \hat{h}_{jk} \quad \forall j \in \mathcal{B}, j > 1, k \in \mathcal{M}, \quad (8)$$

$$\sum_{i \in S} \hat{x}_{ijk} \leq |S| - 1 \quad \forall j \in \mathcal{B}, k \in \mathcal{M}, S \in \mathcal{F}_k, \quad (9)$$

$$\hat{x}_{ijk} \in \{0, 1\}, \hat{y}_{jk} \in \{0, 1\} \quad \forall i \in \mathcal{I}, j \in \mathcal{B}, k \in \mathcal{M}, \quad (10)$$

$$C_{max} \in \mathbb{R}^+, \hat{c}_{jk} \in \mathbb{R}^+, \hat{h}_{jk} \in \mathbb{R}^+ \quad \forall j \in \mathcal{B}, k \in \mathcal{M}, \quad (11)$$

The objective function defined in (1) minimizes the makespan, while constraints (2) set the makespan to be greater than or equal to the maximum completion time of all batches on the available machines. Constraints (3) ensure that each part is assigned to exactly one batch. Constraints (4) link variables $\hat{x}_{ijk}$ and $\hat{y}_{jk}$ and establish that batch j is processed on machine k if any part i is assigned to this batch. Inequalities (5) determine the height of a batch on a machine $\hat{h}_{jk}$ to the maximum height of any part i assigned to it. A subset S_k^{IA} stores the infeasible assignments of part $i \in \mathcal{I}$ to printer $k \in \mathcal{M}$. We obtain set S_k^{IA} in preprocessing by checking for all parts whether a part exceeds the width, length, or height of the build space of k . In this check, the possibility of rotating a part 90 degrees around the z-axis is also taken into account. Consequently, (6) prohibits the assignment of items that have already been proven infeasible for placement on the specific machine. The completion times $\hat{c}_{jk}$ of batch j on machine k are defined by (7) and (8). According to the specifications laid out in Section 3.2, the processing time of a batch is determined by the setup time, the summed scan time to produce the total volume of the batch, and the time spent to cover the batch height with powder. With (8), we take into account the completion times of previous batches on the same machine. In (9), we use combinatorial Benders cuts (Codato and Fischetti, 2006) to exclude infeasible subsets of the sub-problem that is a 2D-OPR. A subset S is infeasible if we can prove that there is no non-overlapping placement of all parts $i \in S$ within the boundaries of

Branch-and-cut for integrated planning in additive manufacturing

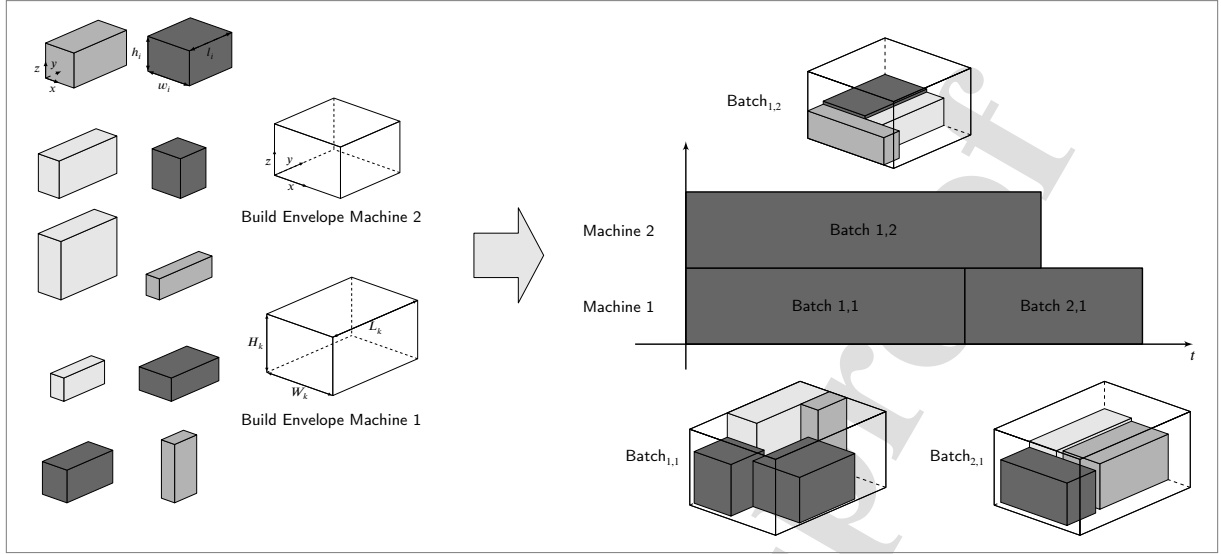


Figure 2: Example problem instance and its solution for the considered problem

machine k . Constraints (9) enforce the exclusion of subset S from being assigned to a single batch j on machine k . This is achieved by limiting the maximum number of parts allocated together within a single batch to $|S| - 1$ (see, e. g., Côté et al., 2021). Variable domains are given in (10)–(11).

For our model, we adopt the formulation of the unrelated machine environment used in Kucukkoc (2019). A new feature of the model is the replacement of area constraints by combinatorial Benders cuts (9). Because Kucukkoc (2019) focuses explicitly on scheduling for AM, the packing problem is relaxed by implementing area restrictions, allowing assignments only if the summed area of assigned parts is smaller than or equal to the machine's build area. In contrast to this approach, we use the generic formulation adopted from the recent paper of Côté et al. (2021) on the 2D-BPP. Because the evaluation of a given subset S being part of $\mathcal{F}_k$ is NP-complete (Clautiaux et al., 2007a), it is a very challenging task. In the following section, we illustrate our solution approach to the considered problem. In addition, we describe the detailed steps to determine whether a subset $S \in \mathcal{F}_k$ and therefore must be excluded from the solution space.

4. A branch-and-cut approach

4.1. Overall solution procedure

Figure 3 illustrates the proposed solution method. We consider (1)–(11) as the master problem, wherein no constraints of type (9) are initially imposed. To strengthen the formulated master problem, we conduct several preprocessing steps, apply additional constraints, and construct an initial solution for the problem instance. A detailed description of these improvements can be found in Section 4.2 and Section 4.3.

Subsequently, the algorithm solves the problem in a branch-and-cut manner by iteratively adding cuts to the model if necessary. This approach is also denoted as logic-based Benders decomposition in the literature (see, e. g., Hooker and Ottosson, 2003; Delorme et al., 2017), and has been recently applied to similar problem configurations (Mao et al., 2024). For each solution candidate, we derive a distinct allocation of parts to batches and machines using the values of variables $\hat{x}_{ijk}$. We validate this current solution with regard to the feasibility of the packed parts in each batch. To this end, we use a step-wise verification mechanism that combines several state-of-the-art techniques from the packing literature. We recombine and extend these techniques to account for an efficient solving of the 2D-OPR. To the best of our knowledge, this is the first time a logic-based decomposition algorithm is proposed for the integrated planning of scheduling on unrelated parallel machines and rectangular packing in AM. Section 4.4 provides detailed explanations of the strategies and their implementation. If an infeasible packing is found, we add a cut to the model and proceed

Branch-and-cut for integrated planning in additive manufacturing

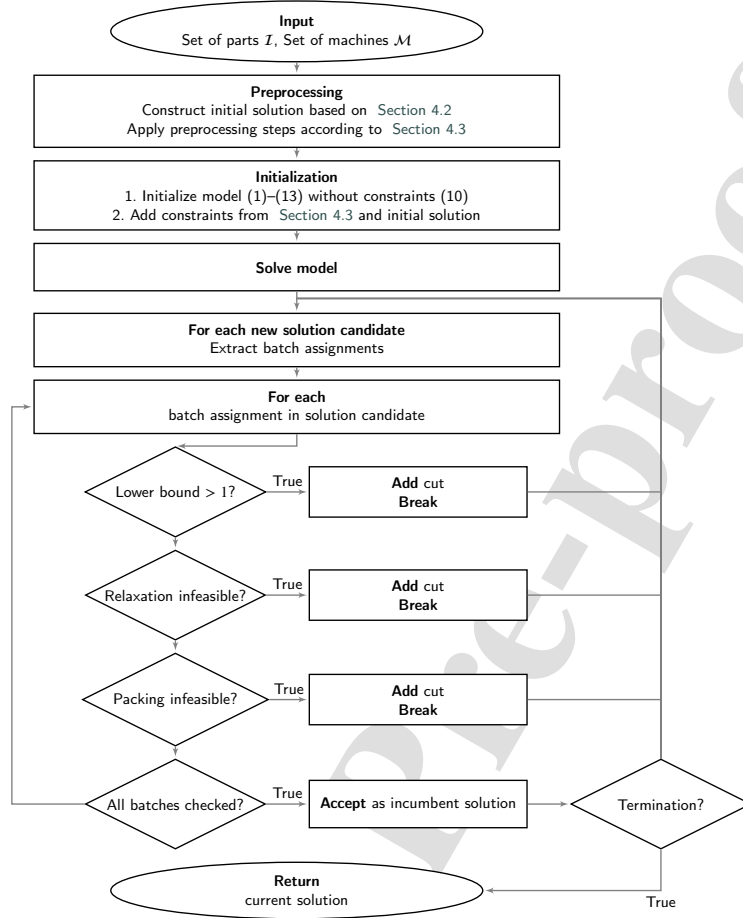


Figure 3: General Solution Strategy

with solving. If no violations can be identified for all used batches, the solution is feasible. The algorithm terminates if the optimal solution is found or if the time limit is reached.

4.2. Construction of initial solutions

To obtain a start solution, we use the principles of the first-fit decreasing heuristic by Johnson (1973) with the extension by shelves (Baker and Schwarz, 1983; Bezerra et al., 2020). Figure 4 summarizes the procedure of finding a start solution for a given instance.

The construction phase begins with initializing a solution pool to store obtained solutions. Subsequently, we determine the allocations between parts and available machines, whereby we aim to balance the workload between the printers: given a set of parts $\mathcal{I}$ and a set of machines $\mathcal{M}$, the algorithm first determines the possible machine assignments for each part using the set of infeasible part-machine assignments S_k^{IA} . Next, we randomly allocate each part to a feasible machine resulting in a permutation with repetition. This allocation is improved using a simple local search with a neighborhood that exchanges the machine assignment for one specific part at a time. Neighboring solutions are evaluated by calculating machine workloads as if all parts fit into one batch, using the maximum workload as the criterion, without considering part dimensions. The evaluation criterion is the maximum among all workloads; thus, we do not consider the dimensions of the parts in this step.

Once an improved allocation is found, we generate solutions using several sorting rules stored in $\mathcal{R}$. Set $\mathcal{I}$ is reordered in accordance with the respective rule. Next, a solution s is generated by applying a shelf first-fit heuristic to form

Branch-and-cut for integrated planning in additive manufacturing

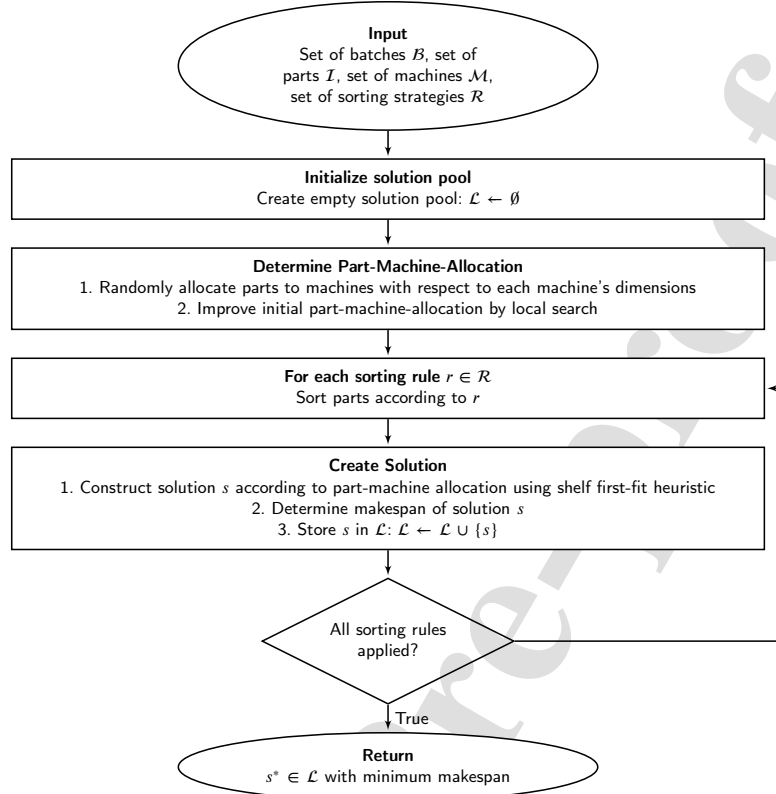


Figure 4: Constructing initial solutions

batches from the reordered parts. This procedure assigns parts to the first feasible batch on a respective machine, where the feasibility of the packing is checked using the shelf packing algorithm of Baker and Schwarz (1983). All potential start solutions are stored in set $\mathcal{L}$. The construction phase concludes by returning the solution s^* with the minimum makespan.

4.3. Model improvements

4.3.1. Reducing the machine sizes

Before initiating the solution process, we employ a technique to reduce the machine sizes. For simplicity, we omit the machine index k in the following discussion, since the modifications apply to all machines. If any combination of the given parts cannot produce the machine's width W (resp. length L), then we are able to shrink the width (resp. length) without losing optimality. Consequently, W can be set to a maximum value W^* , where

$$W^* = \max \left\{ z = \sum_{i \in \mathcal{I}} \hat{z}_i \cdot w_i, z \leq W, \hat{z}_i \in \{0, 1\}, i \in \mathcal{I} \right\}. \quad (12)$$

For the 2D-SPP, this is first presented by Alvarez-Valdés et al. (2009), and since then it has been incorporated in several approaches throughout the packing literature (see, e. g., Delorme et al., 2017). We compute W^* by solving a modified subset sum problem that accounts for rotatable parts using dynamic programming, and subsequently reduce W to W^* . The described technique is applied to both the width and the length of each machine in the problem instance.

4.3.2. Additional constraints

We extend the presented model with valid inequalities and fix variables to accelerate the solution process. First, we augment our model by incorporating

$$\hat{h}_{jk} \leq \max_{i \in I} \{h_i\} \cdot \hat{y}_{jk} \quad \forall j \in B, k \in \mathcal{M}. \quad (13)$$

These constraints are classical big-M constraints linking continuous variables $\hat{h}_{jk}$ and binary variables $\hat{y}_{jk}$. In our case, they restrict batch heights to zero if a batch is not in use. These constraints are not necessarily needed for a valid model formulation; however, they can improve the performance of the model. We further allow a total of b batches to be assigned to each available machine. If b' denotes the number of used batches on a specific machine, then the total number of possible choices for used batches is $\binom{b}{b'}$ (Che et al., 2021). To break this symmetry, we allow a batch j to be opened only if its predecessor $j - 1$ on the same machine is also used; we do so by adding inequalities

$$\hat{y}_{j-1,k} \geq \hat{y}_{jk} \quad \forall j \in B, j > 0, k \in \mathcal{M}, \quad (14)$$

to the model of the master problem. Similar constraints can be found in recent relevant literature (see, e. g., Kucukkoc, 2019; Che et al., 2021). We can further reduce symmetry by prohibiting the assignment of part i to batch j if $j > i$ -i. e., part 1 is only allowed to be assigned to batch 1, while part 2 can be placed in batches 1 and 2, and so on. We adapt this specification for our problem of unrelated machines based on Côté et al. (2021) with

$$\hat{x}_{ijk} = 0 \quad \forall j \in B, i \in I, j > i, k \in \mathcal{M}. \quad (15)$$

Constraints (16) limit the total area of all parts assigned to a batch j to be less than or equal to the available area A_k of machine k . These inequalities are already used to model one-dimensional packing based on area (see, e. g., Kucukkoc, 2019; Rohaninejad et al., 2021). In the current problem configuration, it helps in significantly reducing the number of checks in the sub-problem, since the verification mechanisms only need to be called for solution candidates which satisfy the continuous lower bound $LB_0 \leq 1$ for all batches, where LB_0 is defined as $LB_0 = \lceil \sum_{i \in S} a_i / A_k \rceil$.

$$\sum_{i \in I} \hat{x}_{ijk} \cdot a_i \leq A_k \quad \forall j \in B, k \in \mathcal{M}. \quad (16)$$

Additionally, we define a set of incompatibilities. Let S_{ik}^{IP} be a subset of parts. If a part $i' \in S_{ik}^{\text{IP}}$, this part i' cannot be placed together with part i in the same batch on machine k , since they do not fit together in the build area of k . We determine all parts $i' \in S_{ik}^{\text{IP}}$ in the preprocessing step and propose to add inequalities (17) to the model, allowing only one of the parts in the same batch j on machine k . Note that incompatibilities must be defined individually for each machine, due to the machines being unrelated.

$$\hat{x}_{ijk} + \hat{x}_{i'jk} \leq 1 \quad \forall j \in B, i \in I, k \in \mathcal{M}, i' \in S_{ik}^{\text{IP}}. \quad (17)$$

4.3.3. Reducing the number of available batches

In Section 3.2, we assume $n = b$ on each machine in order to ensure feasibility. This assumption leads to many unnecessary decision variables that need to be handled and thus to increased computational effort. We reduce the total batch number b with a heuristic procedure based on the first-fit decreasing height heuristic (Chung et al., 1982; Berkey and Wang, 1987). Given a set of parts I and a set of machines $\mathcal{M}$, the algorithm splits the studied problem into individual single machine problems, and for each machine $k \in \mathcal{M}$, we run the same steps, as follows. First, all parts that cannot be placed within the build area of k are discarded. Then, all remaining parts are rotated so that $w_i \geq l_i$ and are sorted by decreasing part width. After that, we fill batches according to first-fit for the respective machine until all parts are assigned. We store the number of needed batches b_k for machine k . Finally, b is set to the maximum of all b_k -i. e., $b = \max_{k \in \mathcal{M}} \{b_k\}$. Since we use the maximum of all machine-specific upper bounds on the batch numbers, this reduction decreases the number of decision variables $\hat{x}_{ijk}$ and $\hat{y}_{jk}$ while preserving optimality of solutions. The detailed proofs are given in the supplementary material to this study.

4.4. Evaluation of solution candidates

Whenever a solution candidate is found for model (1)–(11), its feasibility must be verified by checking each of its nonempty batches. Each nonempty batch of a solution consists of a specific subset of parts $S \subseteq \mathcal{I}$. If any of these checks reveal that the subset S is infeasible for its associated machine k , we add no-good cuts (18) for all batches j of this specific machine k . These cuts exclude infeasible solutions from the solution space.

$$\sum_{i \in S} \hat{x}_{ijk} \leq |S| - 1 \quad \forall j \in B. \quad (18)$$

To guarantee the feasibility or infeasibility of a batch, a 2D-OPR must be solved. Since this is a computationally challenging problem (Côté et al., 2021), we want to avoid unnecessary calls to solve the actual 2D-OPR; instead, we try to prove the infeasibility of a batch as fast as possible. Consequently, we verify a given batch in multiple steps. First, we preprocess the given batch aiming to reduce the complexity of the packing problem. Thereafter, we use lower bound techniques and invoke a relaxation approach based on the non-contiguous bin packing problem (NCBP) (Belov et al., 2009; Boschetti and Montaletti, 2010) to identify infeasible batches early. If we cannot prove the infeasibility of the given batch with these steps, we solve the 2D-OPR with an exact method. The procedures of the verification mechanism are explained in the following subsections. Since these explanations pertain to a specific batch on a specific machine, we omit the machine and batch indices to maintain clarity.

4.4.1. Preprocessing packing instances

In addition to the general preprocessing procedure described in Section 4.3.1, we apply further procedures that are only valid for the verification process of a specific batch S . The first reduction is only applied if the build platform of the designated machine is square; In this case, we look for large parts that completely consume an edge of the platform with either their width or length. If such a part exists, we remove it from S and reduce the machine size in accordance with the remaining part dimension. For example, let us assume a platform of 25x25, and S includes a part i with 25x5: Without the loss of optimality, we can exclude i from S and reduce the platform to 25x20. Secondly, we mirror the procedure outlined in Section 4.3.1. Since we now evaluate the feasibility of a subset $S \subseteq \mathcal{I}$, the maximum value of W^* might be smaller; therefore, we run this procedure again.

The last step considers increasing the item sizes and works in a similar way as the reduction of the machine sizes. This method has already been used in previous studies on various packing problems (see, e. g., Boschetti et al., 2002; Carlier et al., 2007; Côté et al., 2021). We propose the following extension to account for rotatable items: For each item $i' \in S$, we increase the width $w_{i'}$ according to

$$w_{i'}^W = W - \max \left\{ z = \sum_{i \in S \setminus \{i'\}} \hat{z}_i w_i + \hat{z}'_i l_i : z \leq W - w_{i'}, \hat{z}_i + \hat{z}'_i \leq 1, \hat{z}_i, \hat{z}'_i \in \{0, 1\}, i \in S \setminus \{i'\} \right\}, \quad (19)$$

$$w_{i'}^L = L - \max \left\{ z = \sum_{i \in S \setminus \{i'\}} \hat{z}_i w_i + \hat{z}'_i l_i : z \leq L - w_{i'}, \hat{z}_i + \hat{z}'_i \leq 1, \hat{z}_i, \hat{z}'_i \in \{0, 1\}, i \in S \setminus \{i'\} \right\}, \quad (20)$$

$$w_{i'} = \min \{ w_{i'}^W, w_{i'}^L \}. \quad (21)$$

We use variables $\hat{z}_i$ and $\hat{z}'_i$ to indicate, whether an item $i \in S \setminus \{i'\}$ is placed in its original orientation or has been rotated. Since the item i' to be enlarged is also able to rotate, we need to consider both machine dimensions with (19) and (20), which we solve using dynamic programming. The same procedure is used to enlarge the length $l_{i'}$ of item i' .

4.4.2. Infeasibility by lower bounds

After preprocessing the packing instance S , the first step of our verification mechanism checks whether a batch is feasible by computing its lower bound on the number of needed batches. We use the bound LB_{DMV} presented in Dell'Amico et al. (2002), which is based on first cutting the parts into squares to make rotations redundant. This cutting

procedure creates set $\mathcal{J}$ of square parts from S . Several subsets are built depending on threshold q , so that $0 \leq q \leq \frac{L}{2}$ and the edge length of each part i' given by $l_{i'}$, $i' \in \mathcal{J}$. Another potential bound, LB_{BM} , is presented in Boschetti and Mingozzi (2003); in contrast to LB_{DMV} , this bound explicitly takes into account the possible rotation of parts. Due to the time complexity of $O(n^3)$ for LB_{BM} in comparison to $O(n')$ for LB_{DMV} with $n = |S|$, $n' = |\mathcal{J}|$, we decided to only use LB_{DMV} (Boschetti and Mingozzi, 2003; Dell'Amico et al., 2002).

In addition to the aforementioned bounding techniques, we propose another bound based on DFFs. We denote this bound as LB_{PM} , as it adapts the ideas presented in Polyakovskiy and M'Hallah (2018). A DFF represents a function $f : [0, 1] \rightarrow [0, 1]$, in which for any set $\mathcal{X}$ of non-negative real numbers, there is a relation in accordance with expression (22) (Fekete and Schepers, 2004).

$$\sum_{x \in \mathcal{X}} x \leq 1 \implies \sum_{x \in \mathcal{X}} f(x) \leq 1. \quad (22)$$

Given two DFFs f_1 and f_2 , we are able to transform the scaled width w_i^* and length l_i^* of part $i \in S$ into $(f_1(w_i^*), f_2(l_i^*))$, where $w_i^* = w_i/W$ and $l_i^* = l_i/L$. Let a_{ci}^* be a scaled area, where $a_{ci}^* = f_1(w_i^*) \cdot f_2(l_i^*)$ and c describes a specific combination of f_1 and f_2 . A feasible packing is only possible if the sum of all scaled areas is less than or equal to 1 (Polyakovskiy and M'Hallah, 2018), i.e.,

$$\sum_{i \in S} a_{ci}^* \leq 1. \quad (23)$$

To account for rotatable parts, let $S^E = \{1, \dots, i, \dots, n, n+1, \dots, 2n\}$ be the set of parts in S augmented by the identical parts rotated by 90 degrees ($n+1, \dots, 2n$). Set $\mathcal{C}$ contains several combinations of DFFs f_1 and f_2 . In this study, we apply the functions $u^{(e)}$, $U^{(e)}$, and $\phi^{(e)}$ along with their combinations presented in Fekete and Schepers (2004) and Fekete et al. (2007). A valid lower bound LB_{PM} for a given subset S is then defined by

$$LB_{PM} = \max_{c \in \mathcal{C}} \left\{ \sum_{i \in S^E} \min\{a_{ci}^*, a_{c,n+i}^*\} \right\}. \quad (24)$$

If the resulting value for LB_{DMV} or LB_{PM} is greater than 1, we have found an infeasible batch S and therefore add a no-good cut (18) to the master problem (1)–(11); otherwise, the algorithm continues with the next check.

4.4.3. Infeasibility by orthogonal relaxation

In the second step, we try to prove the infeasibility of subset S using a relaxation procedure based on the 1CBP. In the 1CBP, each item i is cut into w_i slices of length l_i and width 1, and the bin size transforms into length L and width 1. The goal is to pack all slices into the minimum number of bins, under the condition that all slices of the same part i must be packed side-by-side with each other (Côté et al., 2014). This condition is omitted for the NCBP, which is also known as the bar relaxation (Belov et al., 2009). To prohibit the stacking of slices of the same part $i \in \mathcal{I}$, we require that at most one slice of each part is packed into the same bin.

The NCBP can be described as follows. A pattern t identifies a subset of parts whose summed length is less than or equal to the machine's length. Originally, the pattern is described by an array $(\sigma_{1t}, \dots, \sigma_{it}, \dots, \sigma_{nt})^T$, where σ_{it} is 1 if part i is in the pattern, and 0 otherwise (Côté et al., 2014). To account for the possible rotation of a part, we additionally allow σ_{it} to take the value of w_i/l_i if the part is rotated in the pattern. Let $\mathcal{T}$ be the set of all currently available patterns that include at least one slice of an item, and let integer variable $\hat{u}_t$ denote how often a pattern t is used in the solution. Then, in accordance with Côté et al. (2014), the NCBP can be modeled as:

$$\sum_{t \in \mathcal{T}} \hat{u}_t \rightarrow \min \quad (25)$$

s.t.:

$$\sum_{t \in \mathcal{T}} \sigma_{it} \cdot \hat{u}_t \geq w_i \quad \forall i \in S, \quad (26)$$

Branch-and-cut for integrated planning in additive manufacturing

$$\hat{u}_t \in \mathbb{N}_{\geq 0} \quad \forall t \in \mathcal{T}. \quad (27)$$

In (25), we minimize the total number of used patterns. Constraints (26) ensure that the total number of appearances of each part i in all used patterns is greater than or equal to the part's width w_i . (27) defines the domain of the decision variable $\hat{u}_t$. Since the NCBP is an NP-hard problem, we follow Côté et al. (2014) and only consider the continuous relaxation of (25)–(27), for which we use the column generation approach presented by Gilmore and Gomory (1961) for the cutting stock problem. $\mathcal{T}$ is initialized with n patterns, each of which incorporates a slice of one individual part i . In order to take into account the restriction that each pattern t can include only one slice of a part, as well as to account for the possibility to rotate the parts, we adjust the sub-problem of the original column generation approach.

Let $S^E = \{1, \dots, i, \dots, n, n+1, \dots, 2n\}$ again be the set of parts in S augmented by their rotated versions. The binary variable $\hat{s}_i$ is 1 if a part i is in the solution of the sub-problem, and 0 otherwise. Given the dual values π_i of (26), we define the sub-problem as:

$$1 - \left(\sum_{i=1}^n \pi_i \cdot \hat{s}_i + \sum_{i=n+1}^{2n} \frac{l_i}{w_i} \cdot \pi_i - n \cdot \hat{s}_i \right) \rightarrow \min \quad (28)$$

s.t.:

$$\sum_{i \in S^E} \hat{s}_i l_i \leq L, \quad (29)$$

$$\hat{s}_i + \hat{s}_{i+n} \leq 1 \quad \forall i \in S, \quad (30)$$

$$\hat{s}_i \in \{0, 1\} \quad \forall i \in S^E. \quad (31)$$

Similar to other column generation techniques, we seek to find a new pattern to add to the master problem, which we do by finding a combination of part slices that minimizes the objective function (28). Herein, we consider both: non-rotated parts with $i \in \{1, \dots, n\}$ and rotated parts with $i \in \{n+1, \dots, 2n\}$. To account for the rotation of part $n+i$ in the objective function, the dual value π_i of part i is multiplied by the ratio $l_i + n/w_i + n$. Note that $l_i + n/w_i + n = w_i/l_i$, since part $n+i$ represents the rotated version of part i . We ensure that the summed length of slices does not exceed the length of the machine with (29). Constraints (30) enforce that only one rotation for each part is used. Whenever the sub-problem is solved, we extract the found solution and create a new pattern to add to the master problem. As stated above, if a part i appears in the solution with its original orientation, we add 1 to the new pattern; in cases in which part $n+i$ appears in the solution, the value of part i in the considered pattern of the master problem is w_i/l_i . The interested reader is referred to the supplementary material of this work, where we have provided additional information on the principles of this orthogonal relaxation technique.

We continue the search for new patterns until no better solution can be found. In this case, we compare the continuous lower bound of model (25)–(27) with the width of the machine to which the examined batch has been assigned. If the value of the lower bound is larger than the machine's width, we have found an infeasible subset S ; at this point, we add a cut of type (18) and continue the solution process of model (1)–(11).

4.4.4. Infeasibility by orthogonal packing

In cases in which the above-mentioned verification steps are not able to prove the infeasibility of a batch, we solve the 2D-OPR exactly to reach a final decision about the infeasibility or feasibility of the batch. To this end, we opt for a CP model, which can be solved by a commercial solver. While tailored algorithms for solving the 2D-OPR exist (see, e. g., Kenmochi et al., 2009), we follow the idea of Delorme et al. (2017) and create a non-tailored algorithm, which is easy to replicate and is programming language-agnostic. Moreover, CP has already been proven to be suitable for two-dimensional orthogonal packing in several publications (see, e. g., Martello et al. (2007); Pisinger and Sigurd (2007); Clautiaux et al. (2008)); specifically, Polyakovskiy and M'Hallah (2021) use CP to solve the 2D-OPR sub-problem in a similar problem configuration.

To model the 2D-OPR as a constraint satisfaction problem (CSP), we follow the formulation used in Zipfel et al. (2024) and propose enhancement strategies. Variables $\hat{x}_i^S$, $\hat{x}_i^E$, $\hat{y}_i^S$, and $\hat{y}_i^E$ define the start and end positions of a part i on the

Branch-and-cut for integrated planning in additive manufacturing

x-axis and on the y-axis, respectively. The variables $\hat{w}_i$ and $\hat{l}_i$ represent the width and the length of a part i , respectively. We use the interval variables

$$interval_i^x \rightarrow \hat{x}_i^S + \hat{w}_i = \hat{x}_i^E \quad \forall i \in S, \quad (32)$$

$$interval_i^y \rightarrow \hat{y}_i^S + \hat{l}_i = \hat{y}_i^E \quad \forall i \in S, \quad (33)$$

to link the start and end points of part i on an axis with its respective dimension. Furthermore, the binary variable $\hat{e}_{io}$ is 1 if part i is placed with rotation variant $o \in \mathcal{O}_i$, and 0 otherwise. Finally, w_{io} and l_{io} denote the width and the length, respectively, of part i in rotation o . The CP model for the CSP can be formulated as follows:

$$\sum_{o \in \mathcal{O}_i} \hat{e}_{io} = 1 \quad \forall i \in S, \quad (34)$$

$$NoOverlap2D(\{interval_i^x : i \in S\}, \{interval_i^y : i \in S\}), \quad (35)$$

$$\hat{e}_{io} = 1 \Rightarrow \hat{w}_i = w_{io} \wedge \hat{l}_i = l_{io} \quad \forall i \in S, o \in \mathcal{O}_i. \quad (36)$$

Constraints (34) ensure that exactly one orientation for each part is chosen. Constraint (35) is a global constraint and guarantees that all rectangles are non-overlapping. The rectangle associated with part i is defined by the combination of interval variables $interval_i^x$ and $interval_i^y$. Constraints (36) determine the width and length of part i based on the selected rotation variant o . It should be noted that due to the consideration of rectangles and the possibility of rotating a part 90 degrees, the number of possible rotation variants is 1 if the part is represented as a square, and 2 otherwise.

In what follows, we discuss the domains of the variables. Since there are at most two different rotation variants, the variables $\hat{w}_i$ and $\hat{l}_i$ can only take the width and length of the rotated or non-rotated part. Hence, the domains are defined according to

$$\hat{w}_i \in \left\{ \min_{o \in \mathcal{O}_i} \{w_{io}\}; \max_{o \in \mathcal{O}_i} \{w_{io}\} \right\} \quad \forall i \in S, \quad (37)$$

$$\hat{l}_i \in \left\{ \min_{o \in \mathcal{O}_i} \{l_{io}\}; \max_{o \in \mathcal{O}_i} \{l_{io}\} \right\} \quad \forall i \in S. \quad (38)$$

For the positioning variables $\hat{x}_i^S$, $\hat{x}_i^E$, $\hat{y}_i^S$, and $\hat{y}_i^E$, we reduce the solution space of the 2D-OPR by using the principle of minimal meet-in-the-middle patterns (MMIM), introduced by Côté and Iori (2018). To respect the rotation of parts, we adjust the procedures $MinimalMIMSet(I; W)$ and $NormalPatterns(I; W)$ presented by these authors. By adding another loop for rotations $o \in \mathcal{O}_i$ in line 6, Algorithm 1 respects rotations in normal patterns. The resulting procedure is illustrated as $AdjustedNormalPatterns(I; W)$ in Algorithm 1. For $MinimalMIMSet(I; W)$, this is done by invoking $AdjustedNormalPatterns(S \setminus i; W - \min_{o \in \mathcal{O}_i} \{w_{io}\})$, taking into account the rotations on filling the arrays T^{left} , T^{right} , and defining the minimal set of placement points for part $i \in I$. We display the pseudocode for $AdjustedMinimalMIMSet(I; W)$ in Algorithm 2. After the sets of placement points $\mathcal{P}_i^{\min, x}$ and $\mathcal{P}_i^{\min, y}$ are determined, the variable domains are defined as:

$$\hat{x}_i^S = \left\{ p : p \in \mathcal{P}_i^{\min, x} \right\} \quad \forall i \in S, \quad (39)$$

$$\hat{x}_i^E = \left\{ p + w_{io} : p \in \mathcal{P}_i^{\min, x}, o \in \mathcal{O}_i \mid p + w_{io} \leq W \right\} \quad \forall i \in S, \quad (40)$$

$$\hat{y}_i^S = \left\{ p : p \in \mathcal{P}_i^{\min, y} \right\} \quad \forall i \in S, \quad (41)$$

$$\hat{y}_i^E = \left\{ p + l_{io} : p \in \mathcal{P}_i^{\min, y}, o \in \mathcal{O}_i \mid p + l_{io} \leq L \right\} \quad \forall i \in S. \quad (42)$$

Algorithm 1: AdjustedNormalPatterns($\mathcal{I}; W$) in accordance with Côté and Iori (2018)

Input: set of parts $\mathcal{I}$, machine width W **Output:** set of placement points $\mathcal{N}_0$ according to normal pattern principle1 $T \leftarrow [0 \text{ to } W]$: all entries initialized as 02 $T[0] \leftarrow 1$ 3 **foreach** $i \in \mathcal{I}$ **do**4 **foreach** $p = W - \min_{o \in \mathcal{O}_i} \{w_{io}\}$ **to** 0 **do**5 **if** $T[p] = 1$ **then**6 **foreach** $o \in \mathcal{O}_i$ **do**7 **if** $p + w_{io} \leq W$ **then**8 $T[p + w_{io}] \leftarrow 1$ 9 $\mathcal{N}_0 \leftarrow \emptyset$ 10 **foreach** $p = W$ **to** 0 **do**11 **if** $T[p] = 1$ **then**12 $\mathcal{N}_0 \leftarrow \mathcal{N}_0 \cup \{p\}$ 13 **return** $\mathcal{N}_0$

If the CSP (34)–(36) ascertains the infeasibility of S , we add another no-good cut. As noted by Côté et al. (2021), these cuts are usually weak for large sets S ; thus, we prefer finding a minimal infeasible subset (MIS) $S' \subseteq S$ that cannot be packed onto machine k . Using S' instead of S in no-good cuts (18) would lead to a strengthened cut; however, finding an MIS is a challenging task. Therefore, we apply the heuristic approach from Côté et al. (2021) of finding a reduced infeasible subset (RIS). First, the parts in S are sorted by ascending area; then, we iteratively remove the smallest part from the subset and invoke model (34)–(36) with the reduced set S' and a time limit. The procedure continues as long as infeasibility can be proved for S' . If infeasibility cannot be proven within the time limit or if the considered subset is feasible, we reinsert the last removed part i into S' . After heuristically reducing S to S' , we generate a lifted combinatorial Benders cut, following the procedure developed in Côté et al. (2021) for the 2D-OPP. As noted by these authors, lifting the cut may be a time-consuming task, since it requires repeatedly solving a two-dimensional knapsack problem (2D-KP). Therefore, we again opt for a heuristic approach, introducing a time limit on the solution process of 2D-KP.

5. Computational experiments

5.1. Experiment set up

In this section, we give an explanation of the setting of the computational experiments, before we introduce the test data and present the test results in the subsequent sections. We consider two variants of the algorithm:

- **B&C^{Org}**: This is the original approach, described in the previous sections. All described techniques are included in B&C^{Org}.
- **B&C^{TS}**: This variant represents a two-step procedure of B&C^{Org}. In the first step, we try to improve the initial solution by invoking B&C^{Org} with a more restricted version of constraint (16), such that the summed part areas in a batch are at most 90% of the respective machine area. In the second step, we use the improved initial solution as a new start solution for the original approach with 100% occupation allowed.

The B&C^{TS} variant is designed to mitigate the risk of becoming trapped in checking very dense packings that are hard to solve but not likely to be part of an optimal solution in the unrestricted version. We compare these B&C variants to the MIP model presented by Che et al. (2021). To the best of our knowledge, the existing literature only offers mathematical formulations that are very similar to the one proposed in Che et al. (2021) for exactly solving the integrated problem with two-dimensional packing of rectangular shapes and unrelated machines. Consequently, we benchmark our algorithm against this MIP model, referred to hereafter as MIP^{CHZL}. During preliminary testing, we observed some irregularities in the MIP model, as some packing problems with a known feasible solution were declared infeasible; after thorough investigation, we solved these issues by correcting the Big-M values used in the

Algorithm 2: AdjustedMinimalMIMSet($\mathcal{I}; W$) in accordance with Côté and Iori (2018)

Input: set of parts $\mathcal{I}$, machine width W **Output:** minimal set $\mathcal{P}$ of MIM placement points

```

1  $T^{\text{left}}, T^{\text{right}} \leftarrow [0 \text{ to } W]$ : all entries initialized as 0
2 foreach  $i \in \mathcal{I}$  do
3    $\mathcal{N}_i \leftarrow \text{AdjustedNormalPatterns}(S \setminus i; W - \min_{o \in \mathcal{O}_i} \{w_{io}\})$ 
4   foreach  $p \in \mathcal{N}_i$  do
5      $T^{\text{left}}[p] = 1$ 
6     foreach  $o \in \mathcal{O}_i$  do
7       if  $w_{io} < W$  then
8          $T^{\text{right}}[W - w_{io} - p] \leftarrow T^{\text{right}}[W - w_{io} - p] + 1$ 
9   foreach  $p = 1$  to  $W$  do
10     $T^{\text{left}}[p] \leftarrow T^{\text{left}}[p] + T^{\text{left}}[p - 1]$ 
11     $T^{\text{right}}[W - p] \leftarrow T^{\text{right}}[W - p] + T^{\text{right}}[W - (p - 1)]$ 
12  $t^{\min} \leftarrow 1$ 
13  $\min \leftarrow T^{\text{left}}[0] + T^{\text{right}}[1]$ 
14 foreach  $p = 2$  to  $W$  do
15   if  $T^{\text{left}}[p - 1] + T^{\text{right}}[p] < \min$  then
16      $\min \leftarrow T^{\text{left}}[p - 1] + T^{\text{right}}[p]$ 
17      $t^{\min} \leftarrow p$ 
18  $\mathcal{P} \leftarrow \emptyset$ 
19 foreach  $i \in \mathcal{I}$  do
20    $\mathcal{P}_i \leftarrow \emptyset$ 
21   foreach  $p \in \mathcal{N}_i$  do
22     if  $p < t^{\min}$  then
23        $\mathcal{P}_i \leftarrow \mathcal{P}_i \cup \{p\}$ 
24     foreach  $o \in \mathcal{O}_i$  do
25       if  $W - w_{io} - p \geq t^{\min}$  then
26          $\mathcal{P}_i \leftarrow \mathcal{P}_i \cup \{W - w_{io} - p\}$ 
27    $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{P}_i$ 
28 return  $\mathcal{P}$ 

```

model. A detailed description of the MIP^{CHZL} model and the modifications made can be found in the supplementary material to this study.

The B&C approaches are implemented in PYTHON 3.10 using GUROBI 10.0 and GOOGLE OR-TOOLS 9.7. We use the former to model the master problem and the orthogonal relaxation procedures, and we use the latter to implement the CP model of the 2D-OPR. The MIP^{CHZL} is also modeled using PYTHON and GUROBI.

The study is conducted on an AMD EPYC 7513 with 3.0 GHz clock speed and 64 GB RAM. For the computational tests, the maximum number of threads is set to 8, and an overall time limit of 3,600 seconds is defined for the adjusted MIP^{CHZL} model and both B&C variants. To comply with the overall time limit in the B&C approaches, we dynamically set the time limit of the CP model in a batch's feasibility check to the residual runtime (3,600 seconds minus the elapsed time). The evaluation of a potential cut strengthening in B&C is restricted to two seconds. With regard to the $\text{B\&C}^{\text{TS}}$, the maximum time spent in the restricted version of the master problem is set to 10% of the overall computation time, i. e., 360 seconds). To account for the performance variability of MIP solvers (Lodi and Tramontani, 2014), all test runs are repeated three times with different seed values.

5.2. Description of instance data

We evaluate the performance of the approaches using two different datasets. After introducing the datasets, we classify both sets by part sizes, resulting in five distinct classes that are referenced throughout the evaluations.

Branch-and-cut for integrated planning in additive manufacturing

Data^{CHZL} As stated in Section 2, Che et al. (2021) examine a problem configuration for packing and scheduling in AM that is similar to the problem in this paper: in addition to rotation around the z-axis, the authors also allow a subset of orientations around the x- and y-axes. Table 1 presents the given data for an example part. Each part is defined with up to three different orientations, resulting in different parameter values for its width, length, and height, as well as for the needed volume of support structures (support volume). The authors use four different machine types based on the parameters of Kucukkoc (2019) and Li et al. (2017); the types differ in the dimensions of the building platform, setup times, and recoating times S_k^R (see Table 2). Che et al. (2021) define seven classes of instances, ranging from 20 parts and two machines to 500 parts and seven machines. We focus on the instance classes with 20 and 50 parts (class 1 and class 2), which each contain 20 instances. Che et al. (2021) have solved these classes with their integrated MIP model MIP^{CHZL}, whereas the remaining instance classes have only been solved with a simulated annealing approach.

	Width	Length	Height	Support volume
Orientation				
1	6	2	28	10
2	2	28	6	2
3	6	28	2	0

Table 1
Data description of an example part in Data^{CHZL}

Type	W_k	L_k	H_k	S_k^R	S_k^L	S_k^S
1	25.0	25.0	32.5	0.7	0.030864	2
2	40.0	80.0	50.0	0.25	0.030864	1
3	40.0	40.0	40.0	0.14	0.030864	1
4	40.0	60.0	45.0	0.16	0.030864	1

Table 2
Description of machine data in Data^{CHZL}

To adapt the instances of Che et al. (2021) to our problem setting, we fix the orientation for each part in advance by considering the calculation of the completion times of a batch in constraints (7) and (8). Despite minor variations due to different support volumes, the choice of orientation does not affect the volume-dependent aspect of the processing time; however, taller parts directly influence the height of a batch and thus directly influence the processing time. To expedite the printing process, parts should be orientated to maximize the area in contact with the build platform. Therefore for all parts in Data^{CHZL}, we fix the orientation to the one with the smallest height; for the example part in Table 1, this is orientation 3. The respective support volume is added to the total volume v_i .

After preliminary analyses of Data^{CHZL}, we observed that the majority of the considered parts are very small compared to the sizes of the AM machines. In order to investigate, e. g., the influences of different part sizes on the solution process and the solution quality, a test set need to comprise a diverse set of instances. Additionally, a more diversified benchmark dataset that takes into account varying part dimensions may more accurately represent real-world requirements in AM shops with on-demand business models. Consequently, a new benchmark dataset is needed for planning in AM, one that incorporates different part sizes in accordance with the well-known benchmark data for cutting and packing problems (see, e. g., Berkey and Wang, 1987; Martello and Vigo, 1998).

Data^{ZTK} We first define the machine system and the part sizes for the new test data. The structure of the available machines is explained in the supplementary material of this study. Regarding the dimensions of the machines' build spaces, we follow the product data sheets of machine manufacturers (see, e. g., SLM Solutions AG, 2023) and the machine specifications from related research (Kucukkoc, 2019; Che et al., 2021). We randomly draw machine parameters from the intervals shown in the table to generate the process speed parameters; these intervals are also based on manufacturers' product data sheets. We then use the formulas proposed by Aloui and Hadj-Hamou (2021) to calculate scanning and recoating times.

In terms of part sizes, we follow the approach of Martello and Vigo (1998), who define different part types and create classes in which certain types of parts occur more frequently; this allows us to generate a diverse set of parts. Table 3 gives a detailed explanation of the four part types and the classes of the test data. According to the definition, classes 1 and 2 contain rather small parts, while classes 3 and 4 comprise larger parts, and the parts in classes 2 and 3 tend to be more elongated than those in classes 1 and 4. Finally, we define the test instances in Table 4, resulting in 36 configurations and a total of 180 problem instances. For each instance, we ensure that the build space of machine type 1 is included for at least one machine; the remaining machine build spaces are drawn randomly from the available types shown in the supplementary material. In this way, we ensure that each part fits in at least one machine. All test instances are available online via Zipfel (2024).

Branch-and-cut for integrated planning in additive manufacturing

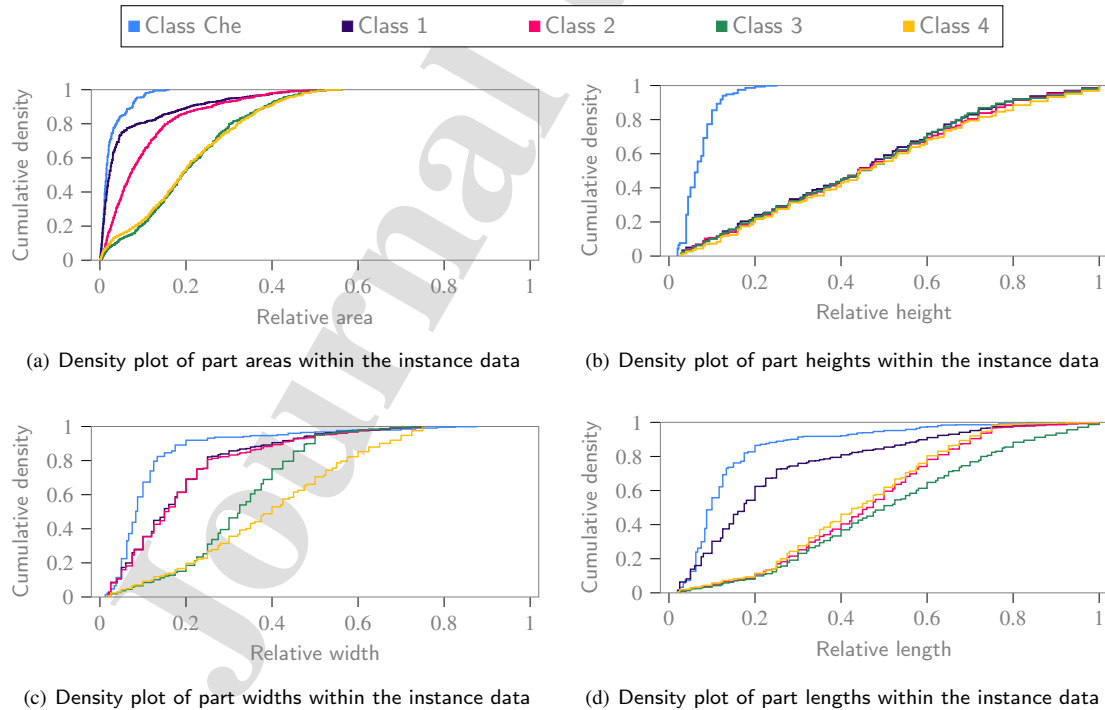
Type 1	random integers: w_i in $[1, \frac{1}{5}W]$, l_i in $[1, \frac{1}{5}L]$, h_i in $[1, \frac{1}{2}H]$
Type 2	random integers: w_i in $[1, \frac{1}{5}W]$, l_i in $[\frac{1}{5}L, \frac{3}{5}L]$, h_i in $[1, \frac{1}{2}H]$
Type 3	random integers: w_i in $[\frac{1}{5}W, \frac{2}{5}W]$, l_i in $[\frac{1}{5}L, \frac{4}{5}L]$, h_i in $[1, \frac{1}{2}H]$
Type 4	random integers: w_i in $[\frac{1}{5}W, \frac{3}{5}W]$, l_i in $[\frac{1}{5}L, \frac{3}{5}L]$, h_i in $[1, \frac{1}{2}H]$
Class 1	part type 1 with probability 70%, type 2, 3, 4 with probability 10% each. $W = L = H = 50$
Class 2	part type 2 with probability 70%, type 1, 3, 4 with probability 10% each. $W = L = H = 50$
Class 3	part type 3 with probability 70%, type 1, 2, 4 with probability 10% each. $W = L = H = 50$
Class 4	part type 4 with probability 70%, type 1, 2, 3 with probability 10% each. $W = L = H = 50$

Table 3Definition of part types and classes for Data^{ZTK}

Param.	Levels	Total
$n \times m$	n	9
	m	
	10	
	20	
	40	
Classes	1, 2, 3, 4	4
	Total number of configurations	
	Number of instances/ configuration	
Number of problem instances	5	180
	Number of runs/ instance	
	3	
Total number of experiments		540

Table 4Problem parameters and their levels for Data^{ZTK}

To underscore the need for new test data, we analyze the part dimensions of the different classes. To this end, we classify Data^{CHZL} as a separate class (class Che) within a combined test set. Figure 5 illustrates the main characteristics of the parts within each class using density plots. As machine sizes vary across the instances, we present relative values based on the maximum individual dimension. Parts of class Che are consistently characterized by rather small part areas and low heights, although the latter is also significantly influenced by the choice of orientation discussed earlier. If a different orientation of the components is chosen, more parts would exhibit increased heights but the part area would simultaneously be further reduced. Classes 1 to 4 are uniformly distributed, as defined in Table 3. While we observe similar relative areas for classes 3 and 4 (see Figure 5(a)), differences in their relative lengths and widths can be seen in Figure 5(c) and Figure 5(d). By augmenting Data^{CHZL} with the new part classes of Data^{ZTK}, we can now comprehensively evaluate the performance of the proposed B&C algorithms across various configurations and scenarios involving different part sizes.

**Figure 5:** Evaluation of item dimensions categorized by item classes

5.3. Computational results

5.3.1. General results

In this section, we present the main results of the computational study. First, we compare the performance of the three exact approaches. Subsequently, we examine the solutions in greater detail to explain the differences in performance. In particular, we highlight the improvements in the upper and lower bounds achieved by the B&C approaches, analyze the different batch numbers needed in a schedule, and show their influence on the objective function values. Based on these analyses, we give key managerial insights that are valuable for practitioners and for future research.

Performance analysis of exact solution approaches for integrated planning in AM

The summarized results of all solution approaches are displayed in Table 5. Tables reporting the test results in more detail can be found in the supplementary material to this study. Detailed output files are available online (Zipfel, 2024). The results are categorized by classes and part numbers n and their respective levels. These levels include individual numbers of problem instances, since we combined both datasets Data^{CHZL} and Data^{ZTK} in the evaluation; accordingly, we highlight the number of runs (#Runs) and instances (#Ins.) for each level. All approaches are compared regarding the number of optimal runs #Opt. and the number of problem instances with at least one optimal run (in brackets), the average optimality gaps G^{avg} in percent, and the average computation time RT^{avg} in seconds. First, we find that the B&C approaches outperform MIP^{CHZL} over all classes. Comparing the summary values of #Opt. and G^{avg} in Table 5, B&C^{Org} and B&C^{TS} demonstrate clear advantages: both variants can prove optimality in at least 60 runs (244 optimal runs by MIP^{CHZL} vs. 304 by B&C^{Org}) and 21 problem instances (84 optimally solved instances by MIP^{CHZL} vs. 105 by B&C^{Org}) where MIP^{CHZL} cannot provide the proven optimal solution. Consequently, the B&C approaches show significantly lower optimality gaps on average. In total, B&C^{Org} and B&C^{TS} reduce G^{avg} by 67.1% and 74.8%, respectively, compared to MIP^{CHZL}. By comparing the number of optimal runs and the number of problem instances with at least one optimal test run, we can observe the performance variability of the commercial solvers; due to the given time limit of 3,600 seconds, the approaches cannot prove optimality in every run.

Table 5

Result summary: Comparing exact solution approaches by class and number of parts

Param.	Level	#Runs	#Ins.	MIP ^{CHZL}			B&C ^{Org}			B&C ^{TS}		
				#Opt.	G^{avg}	RT^{avg}	#Opt.	G^{avg}	RT^{avg}	#Opt.	G^{avg}	RT^{avg}
Class	Che	120	40	72 (24)	1.49	1649.70	72 (25)	2.79	1462.67	77 (26)	0.63	1327.93
	1	135	45	63 (21)	19.78	2086.31	79 (28)	2.20	1603.98	81 (29)	1.43	1567.26
	2	135	45	43 (16)	23.04	2499.51	55 (19)	5.79	2167.55	56 (19)	4.89	2162.22
	3	135	45	34 (12)	19.91	2806.38	48 (16)	7.13	2356.89	51 (17)	6.75	2317.93
	4	135	45	32 (11)	10.58	2863.97	50 (17)	6.92	2378.69	48 (16)	5.07	2398.33
n	10	60	20	60 (20)	0.00	1.58	60 (20)	0.00	1.01	60 (20)	0.00	0.92
	20	180	60	142 (49)	0.46	862.02	179 (60)	0.00	107.53	177 (59)	0.00	126.45
	40	120	40	17 (6)	5.77	3227.12	30 (12)	3.24	2752.83	31 (12)	2.16	2736.05
	50	60	20	12 (4)	2.98	3292.22	12 (5)	5.57	2915.60	17 (6)	1.27	2651.59
	60	120	40	13 (5)	29.19	3447.91	21 (7)	6.10	3031.75	26 (9)	4.16	2957.42
	80	120	40	0 (0)	47.43	3600.44	2 (1)	15.46	3629.02	2 (1)	14.08	3619.97
Total		660	220	244 (84)	15.23	2397.33	304 (105)	5.01	2006.03	313 (107)	3.83	1968.98

#Runs: Number of algorithm runs within each level. #Ins.: Number of problem instances within each class. #Opt.: Number of runs that ended with an optimal solution; number of instances with at least one optimal solution in brackets. G^{avg} : Average MIP gap of all runs within a level in percent. RT^{avg} : Average computation time of all runs within a level in seconds.

Regarding the different classes, we observe that the B&C approaches particularly excel in classes 1 and 2, where part sizes are small. For the class with the smallest parts, class Che, B&C^{Org} results in higher optimality gaps than MIP^{CHZL}; in contrast, B&C^{TS} is better across all metrics. This indicates that the original variant B&C^{Org} tends to get stuck in the exact check of batches very early in the solution process. As a result, the lower and upper bounds cannot improve as effectively as they do in the variant B&C^{TS}. We also find that hard packing instances are not necessarily part of an optimal solution. The intended effect of bypassing some hard packing problems is more pronounced for classes with smaller parts and becomes less relevant when part sizes increase. In addition, we observe a drop in average optimality gaps between classes 3 and 4 for all solution approaches. The part dimensions in xy -plane are more heterogeneous in class 3 than in class 4 resulting in a higher amount of elongated parts in class 3, as shown in Figure 5(c) and Figure 5(d). The impact of this diversity on computation times has been observed in the packing literature (see, e. g., Carlier et al.,

Branch-and-cut for integrated planning in additive manufacturing

2007; Côté et al., 2021). Comparing the different levels of n , we see an expected increase in G^{avg} and RT^{avg} as part numbers grow. However, the MIP^{CHZL} model shows a sharp increase in the optimality gaps as the number of parts increases, while the optimality gaps of both B&C variants grow only moderately. It is noteworthy that the results for $n = 50$, i. e., class Che, are excluded from this trend, as runs of this class typically terminate with smaller gaps than the other classes. In summary, both B&C variants produce superior results to the MIP model for almost all configurations, and the superiority is especially pronounced for larger instance sizes and when problems contain smaller parts.

Detailed analysis of upper and lower bound improvements

In what follows, we examine the results in more detail to determine the reasons for the performance differences between the B&C approaches and MIP^{CHZL} . To this end, Figure 6 provides insights into the improvements in the upper and lower bounds of the B&C variants, using the results of MIP^{CHZL} as the baseline. We focus on the average upper and lower bound for each instance in our analysis, considering only instances with $n \geq 40$ since the results for $n = 10, 20$ are very similar for all approaches.

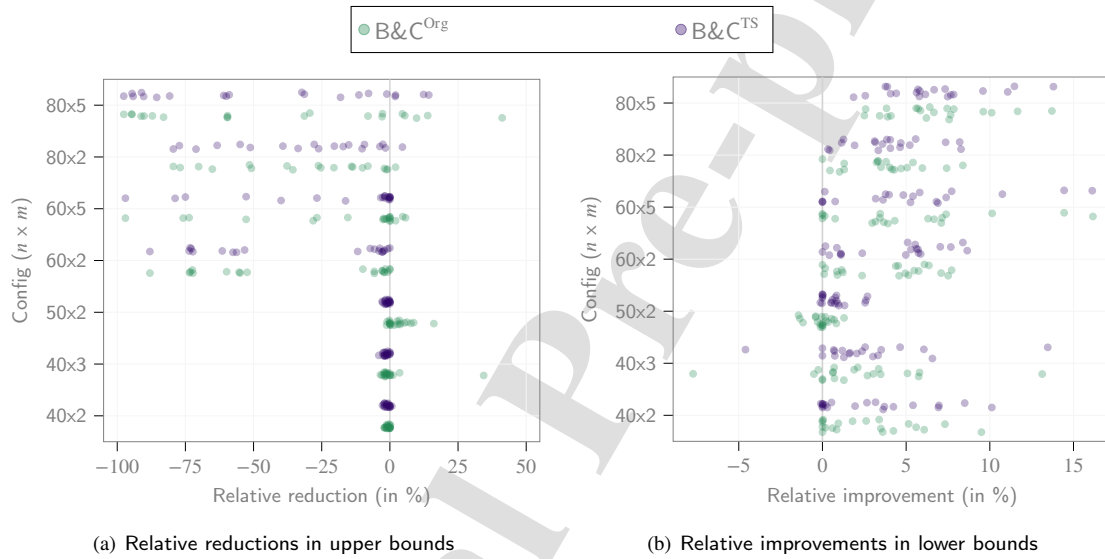


Figure 6: Grouped scatter plots illustrating relative deviations of upper and lower bounds of B&C variants based on MIP^{CHZL} results

Regarding the upper bounds, $\text{B\&C}^{\text{Org}}$ and $\text{B\&C}^{\text{TS}}$ demonstrate average improvements of 16.9% and 18.7%. Additionally, lower bounds can be improved by 3.4% and 3.8% on average. For $n \leq 50$, all approaches find similar upper bounds for the respective problem instances (Figure 6(a)); however, the major advantage of the B&C approaches stems from an improvement of the lower bounds of up to 13.4% for both approaches (Figure 6(b)). Special attention must be paid to level $n = 50$ of $\text{B\&C}^{\text{Org}}$. As $\text{B\&C}^{\text{Org}}$ tends to get stuck in hard packing instances, it generates worse upper and lower bounds in some problem instances, resulting in the worse optimality gaps reported above. In contrast, $\text{B\&C}^{\text{TS}}$ yields better upper and lower bounds than MIP^{CHZL} . The test results on problem instances with $n \geq 60$ show lower and upper bound improvements for both B&C approaches. While the makespan can be reduced by up to 97.6% on some problem instances, the lower bounds can be improved by up to 16.1%. In general, the improvements of $\text{B\&C}^{\text{Org}}$ and $\text{B\&C}^{\text{TS}}$ are very similar, with only a few outliers for $\text{B\&C}^{\text{Org}}$.

Analysis of batch numbers and managerial insights

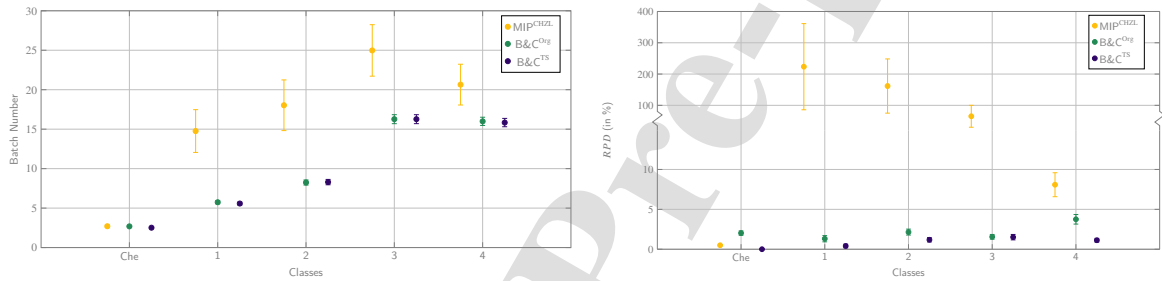
We further analyze the effects of the sophisticated packing methods used in the B&C variants on the resulting schedules and objective values to derive useful managerial insights. To this end, we evaluate the solution approaches across different part classes. Figure 7 presents two key metrics: first, the number of batches from the best-known solutions is reported in Figure 7(a) to explore the effectiveness of the packing methods; second, Figure 7(b) presents the

Branch-and-cut for integrated planning in additive manufacturing

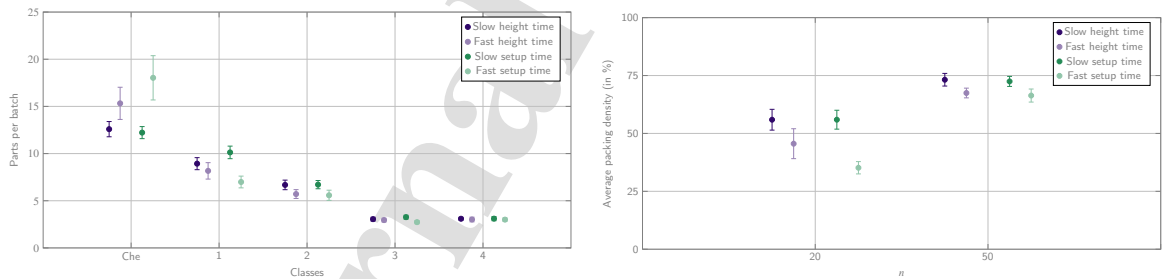
relative percentage deviations ($RPDs$) in objective values, i. e., the makespan. Both figures show mean values and 95% confidence intervals segmented by class level.

Three main observations emerge from these figures. First, there is a clear increase in the total number of batches as part sizes grow from class Che to class 4, due to fewer parts fitting per batch as part sizes increase. Second, the B&C variants demonstrate superior packing efficiency compared to MIP^{CHZL} , particularly for classes 1 to 4, where they show notably lower batch numbers. In class Che, however, the differences are marginal due to the very small part sizes relative to the machine dimensions, which allow both B&C and MIP^{CHZL} to easily minimize the number of batches required (see Figure 7(a)).

Third, the $RPDs$ in Figure 7(b) reveal that, while the makespan differences are minor for class Che, the B&C variants offer substantial improvements for classes 1 and 2, with the mean RPD exceeding 100%. This advantage stems from a higher impact of height- and setup-dependent components on the processing time for these classes, where more effective packing by the B&C variants leads to substantial reductions in processing time. However, as part sizes increase in classes 3 and 4, the influence of these components decreases. In addition, the scheduling subproblem becomes more critical due to the larger number of batches. As a result, the performance gap between the B&C variants and MIP^{CHZL} narrows in these larger classes.



(a) Average batch numbers resulting from all solution approaches by classes (b) Average $RPDs$ resulting from all solution approaches by classes



(c) Average number of parts per batch segmented by slow and fast height and setup times and classes (d) Average packing density in class Che segmented by slow and fast height and setup times

Figure 7: Evaluation of batch numbers, $RPDs$, part numbers per batch, and packing densities categorized by classes

Additionally, we aim to explore how different problem structures, specifically in terms of processing time, influence packing outcomes. Recall that the processing time of a batch is determined by three components: the setup time of the machine, the maximum height to be printed, and the total volume to be produced. While the total volume on a machine can not be reduced through more efficient packing, the packing affects the setup times and the height-dependent processing times. In the following analysis, we investigate the impact of the height-dependent components of the batch processing time in greater detail. We categorize each problem instance based on the average height production time S_k^R and the average setup time S_k^S of the available machines, distinguishing between slow and fast times. The categories are determined using median values. Based on algorithm B&C^{TS}, Figure 7(c) reports the average number of parts allocated per batch across class levels. For class 1, we observe that instances with slower height or setup time tend to pack more

parts into a batch compared to those with faster processing parameters. This trend persists across class levels 1 to 4, although the differences decrease drastically as part sizes increase. For class Che, we see the opposite case, which may be due to the small numbers of batches and the trade-off between volume-dependent and packing-dependent processing time components. However, analyzing the average packing densities in Figure 7(d) we observe higher values for those instances where there the packing-dependent processing is slow due to the characteristics of the available machines. The trend persists for both levels of part numbers. This suggests that packing becomes more critical when the height or setup time constitutes a larger share of the total batch processing time.

We conclude that when packing-dependent time components make up a significant portion of the processing time, sophisticated packing methods are crucial for achieving high-quality schedules. These time components are influenced by two key factors: the dimensions of the parts and the processing speeds of the machines. When there are many flat parts with large volumes the height-dependent time component is less important, and high-quality schedules result from balancing part volumes on the available machines. Conversely, when scheduling tall parts or parts with diverse heights, effective packing impacts the resulting makespan. The same is true for processing speeds of machines. If the height-related processing speed S_k^L are large compared to S_k^R , i. e. the machines are slow in producing height, packing becomes more relevant since the share of height-dependent processing time increases.

Overall, these findings highlight the importance of high-quality packing methods for small to medium-sized parts (classes 1 and 2), where better packings result in fewer batches and significantly reduced makespans. For very small parts (class Che), the B&C methods have less impact on the makespan because the part sizes allow both B&C and MIP^{CHZL} to easily fit many tiny parts into a few batches, minimizing the gains of sophisticated packing. For larger parts (classes 3 and 4), the impact of the B&C methods is also reduced, but for different reasons: here, the scheduling subproblem becomes more critical in determining a high-quality schedule. While the importance of the height-dependent component decreases in the specific problem instances considered, it remains significant in scenarios where part dimensions—height, width, and length—increase proportionally.

5.3.2. Evaluation of algorithmic components

The proposed B&C procedure comprises several improvement techniques, presented in Section 4.2, Section 4.3, and Section 4.4. In what follows, we focus on analyzing the effectiveness of these techniques. We will first examine the model improvements before evaluating the impact of the packing improvements. To this end, we denote B&C^{Basis} as a basic variant of the proposed algorithm. B&C^{Basis} only contains the model of the master problem, given by (1)–(11), and the CP model defined in (34)–(36).

Impact of model improvements

Table 6 compares B&C^{Basis} with the integration of initial solutions based on Section 4.2 and the model improvements introduced in Section 4.3. We observe that B&C^{Basis} cannot generate a feasible solution for all problem instances with $n = 20$. For classes 1 and 2 a feasible solution is always produced, but for the remaining classes a feasible solution can only be obtained for 90% of the runs. Thus, providing an initial solution is a simple approach to impact the test results positively. The initial solution leads to a lower average computation time RT^{avg} in some instances (see, e. g., instances with $n = 20$ and class 1) and more optimally solved instances in class Che. This is because the initial solution can be used by the solver to prune the search tree of the master problem and to find better solutions. When the model improvements are added to the algorithm, we observe a substantial performance improvement of the overall algorithm: The average computation times decrease for all reported configurations, and the number of optimally solved runs increases by 28%, from 177 runs to 227 runs. In summary, the initial solution and the model improvements are highly beneficial for the algorithmic performance of the proposed B&C approach.

Impact of packing improvements

In addition to the model improvements, we evaluate the effectiveness of the proposed packing improvements. More specifically, we categorize the packing improvements into two groups for our comparison. The first category includes the reductions and packing approximations LB_{DMV} , LB_{PM} , and the adjusted bar relaxation, as explained in Section 4.4.3. The second category encompasses the potential cut lifting as detailed at the end of Section 4.4.4. Thus, this category contains all algorithmic improvements and is equivalent to B&C^{Org}. Table 7 summarizes the results. As the baseline, we use the algorithm variant incorporating the initial solution and the model improvements. Adding the packing preprocessing and the approximations further improves the performance of the B&C algorithm. In general,

Branch-and-cut for integrated planning in additive manufacturing

Table 6

Result summary: Evaluation of model improvements on small problem instances by number of parts and class

n	Class	#Runs	B&C ^{Basis}				+ initial solution				+ model improvements			
			#Feas.	#Opt.	G^{avg}	RT^{avg}	#Feas.	#Opt.	G^{avg}	RT^{avg}	#Feas.	#Opt.	G^{avg}	RT^{avg}
10	1	15	15	15 (5)	0.00	0.54	15	15 (5)	0.00	0.60	15	15 (5)	0.00	0.24
	2	15	15	15 (5)	0.00	0.32	15	15 (5)	0.00	0.48	15	15 (5)	0.00	0.35
	3	15	15	15 (5)	0.00	7.58	15	15 (5)	0.00	5.94	15	15 (5)	0.00	0.66
	4	15	15	15 (5)	0.00	12.35	15	15 (5)	0.00	15.64	15	15 (5)	0.00	0.63
20	Che	60	54	54 (18)	0.00	403.48	60	60 (20)	0.00	3.37	60	60 (20)	0.00	1.05
	1	30	30	30 (10)	0.00	36.46	30	30 (10)	0.00	23.92	30	30 (10)	0.00	2.13
	2	30	30	21 (7)	1.07	1099.59	30	21 (7)	1.06	1095.32	30	30 (10)	0.00	33.50
	3	30	27	0 (0)	6.00	3600.48	30	0 (0)	6.18	3600.85	30	24 (8)	0.15	1051.81
	4	30	26	6 (2)	5.91	2888.04	30	6 (2)	5.92	2884.23	30	23 (8)	0.29	1121.51
Total		240	227	171 (57)	1.61	1055.24	240	177 (59)	1.64	952.80	240	227 (76)	0.06	276.50

#Runs: Number of runs within each class. #Feas: Number of runs that ended with a feasible solution. #Opt.: Number of runs that ended with an optimal solution; number of instances with at least one optimal solution in brackets. G^{avg} : Average MIP gap of all runs within a class in percent. RT^{avg} : Average computation time of all runs within a class in seconds.

more runs and instances can be solved optimally, and the average runtime across all instances decreases from 276.50 seconds to 172.46 seconds. However, upon investigating the results for the individual classes, we observe a decrease in computational effort only for classes 2, 3, and 4 of $n = 20$. For problem instances with fewer and smaller parts, the results show a slight increase in computation time. Similar trends emerge when cut lifting is enabled: cut lifting further increases the number of runs with proven optimality and reduces the average computation time; however, the procedure is very time-consuming in relation to the total computation time for instances with few and small parts, which can negatively impact the overall computing time.

Table 7

Result summary: Evaluation of packing improvements on small problem instances by number of parts and class

n	Class	#Runs	+ model improvements				+ packing approximations				+ cut lifting (B&C ^{Orig})			
			#Feas.	#Opt.	G^{avg}	RT^{avg}	#Feas.	#Opt.	G^{avg}	RT^{avg}	#Feas.	#Opt.	G^{avg}	RT^{avg}
10	1	15	15	15 (5)	0.00	0.24	15	15 (5)	0.00	0.40	15	15 (5)	0.00	0.50
	2	15	15	15 (5)	0.00	0.35	15	15 (5)	0.00	0.57	15	15 (5)	0.00	0.71
	3	15	15	15 (5)	0.00	0.66	15	15 (5)	0.00	0.71	15	15 (5)	0.00	1.49
	4	15	15	15 (5)	0.00	0.63	15	15 (5)	0.00	0.75	15	15 (5)	0.00	1.32
20	Che	60	60	60 (20)	0.00	1.05	60	60 (20)	0.00	3.62	60	60 (20)	0.00	9.74
	1	30	30	30 (10)	0.00	2.13	30	30 (10)	0.00	3.64	30	30 (10)	0.00	3.68
	2	30	30	30 (10)	0.00	33.50	30	30 (10)	0.00	15.86	30	30 (10)	0.00	39.87
	3	30	30	24 (8)	0.15	1051.81	30	27 (9)	0.08	477.87	30	30 (10)	0.00	71.81
	4	30	30	23 (8)	0.29	1121.51	30	28 (10)	0.02	873.87	30	29 (10)	0.01	510.31
Total		240	240	227 (76)	0.06	276.50	240	235 (79)	0.01	172.46	240	239 (80)	0.00	80.90

#Runs: Number of runs within each class. #Feas: number of runs that ended with a feasible solution. #Opt.: Number of runs that ended with an optimal solution; number of instances with at least one optimal solution in brackets. G^{avg} : Average MIP gap of all runs within a class in percent. RT^{avg} : Average computation time of all runs within a class in seconds.

In the following, we further analyze the algorithmic enhancements in greater detail. Table 8 illustrates the impact of the cut lifting procedure. We extract three runs that did not result in an optimal solution with the variant without cut lifting, and we compare them with their equivalent seed runs with the lifting procedure included. We show in column #Cuts the total number of cuts added to the master problem. Columns RT^{PA} , RT^{OP} , RT^{SL} report the time spent in the algorithm parts preprocessing and approximation, orthogonal packing, and in strengthening and lifting the cut, while column RT^{TC} shows the total computation time. It is evident that substantially fewer cuts must be added to the master problem if cut lifting is enabled. This comes with the cost of longer times needed for the cut lifting procedure. However, this is compensated for with stronger cuts and a greatly reduced total computation time.

Impact of the packing approximations on infeasibility detection across problem classes

In Table 9, we present the detailed results on the different packing approximations used in the algorithm. We illustrate

Branch-and-cut for integrated planning in additive manufacturing

Table 8

Result summary: Impact analysis of cut lifting procedures based on selected problem instances

Name	Run	without cut lifting					with cut lifting				
		#Cuts	RT^{PA}	RT^{OP}	RT^{SL}	RT^{TC}	#Cuts	RT^{PA}	RT^{OP}	RT^{SL}	RT^{TC}
Class3-Parts20-Machines2-Instance4	3	1672	1.78	1.97	10.61	3600.40	148	0.28	0.71	33.82	221.34
Class4-Parts20-Machines2-Instance3	2	1108	5.44	2.53	11.27	3600.68	390	2.02	1.95	91.88	199.60
Class4-Parts20-Machines2-Instance5	3	742	10.37	3.92	15.81	3600.40	384	3.50	3.13	62.81	1328.21

#Cuts: Number of cuts generated during the run of the problem instance. RT^{PA} , RT^{OP} , RT^{SL} : Accumulated time consumed by preprocessing and approximation (RT^{PA}), orthogonal packing (RT^{OP}), and cut strengthening and lifting (RT^{SL}) within the sub-problem. RT^{TC} : Total computation time of the run of the problem instance. All times are in seconds.

the effectiveness of each individual mechanism through its contribution to the overall number of detected infeasible packing problems. The columns report the following: #Checks gives the average number of checks per instance in each class, while %Inf. reports the share of proven infeasible checks in percent. The remaining four columns per B&C variant represent the share of the respective procedure in all checks resulting in infeasibility: Columns LB_{DMV} and LB_{PM} present the contribution of the lower bounds, column BR represents the bar relaxation, and column OP is the original 2D-OPR. Our analysis reveals that the average number of checks in the B&C^{TS} variant decreases by about 50% in each class compared to B&C^{Org}. Furthermore, the number of checks increases as the size of the parts grows, because a single solution consists of more batches that must be evaluated. Regarding the different procedures, LB_{DMV} is more successful in proving infeasibility when part sizes grow. The same is true for LB_{PM} for classes Che, 1, and 2; however, when parts get larger in classes 3 and 4, the share of LB_{PM} decreases again. This may be due to an interaction with the growing importance of LB_{DMV} in these classes. The adjusted bar relaxation shows a higher impact on classes with small parts. The share of the final check mechanism decreases as part sizes grow since the preceding checks are particularly useful to detect infeasibility when part sizes are larger. In summary, we find that all considered packing approximations are relevant and contribute to the performance of the proposed algorithm.

Table 9

Result summary: Evaluation of the efficiency of different check procedures

Class	B&C ^{Org}						B&C ^{TS}					
	#Checks	%Inf.	%Inf. by				#Checks	%Inf.	%Inf. by			
			LB_{DMV}	LB_{PM}	BR	OP			LB_{DMV}	LB_{PM}	BR	OP
Che	18.42	4.67	25.58	9.30	6.98	59.30	7.69	6.63	35.29	3.92	11.76	49.02
1	161.33	37.30	13.53	10.75	12.58	63.14	84.45	36.55	14.22	10.20	12.80	62.81
2	394.06	68.03	34.78	13.60	5.60	46.02	163.64	66.63	27.86	13.56	6.17	52.41
3	848.24	66.53	69.78	7.90	6.33	15.99	393.07	48.14	65.27	8.86	6.87	19.01
4	908.27	62.97	71.76	10.68	5.60	11.96	478.39	48.45	68.33	11.51	6.31	13.85

#Check: Average number of checks conducted within each class. %Inf.: Share of proven infeasible checks in percent. LB_{DMV} , LB_{PM} , BR, OP: Share of infeasible packings detected by lower bounds (LB_{DMV} , LB_{PM}), adjusted bar relaxation (BR), and final orthogonal packing (OP) based on the total number of infeasible packings.

Solving the orthogonal packing problem with constraint programming

Lastly, we concentrate on the CP model employed to solve the 2D-OPR in the final verification step. Figure 8 presents aggregated results of two key characteristics we extracted from the experiments. Figure 8(a) illustrates the empirical cumulative distribution function of the packing problems solved by the CP model over time. In the computational study, the variants B&C^{Org} and B&C^{TS} made a total of 91,024 and 47,096 calls to the CP model, respectively. Both variants can determine the status—feasible or infeasible—in 99.8% of the cases. Figure 8(a) reveals several interesting insights. The proposed CP model is highly effective in proving the feasibility of a packing problem. The 90% percentile of the time required to prove feasibility is less than 0.05 seconds for both variants. The time needed to prove infeasibility is slightly slower, with the 90% percentiles at around 0.25 seconds. Regardless, within the first 0.5 seconds of computation time, both B&C variants solve over 95% of all encountered packing problems.

We further investigate hard packing instances for which the CP model fails to determine the status in a reasonable time. Following Clautiaux et al. (2007a), we calculate measure ϵ for each packing problem with the set of parts S on a machine k that has a runtime of at least 300 seconds, where $\sum_{i \in S} a_i = (1 - \epsilon) \cdot A_k$, meaning that ϵ measures the free

Branch-and-cut for integrated planning in additive manufacturing

area when considering a particular batch. Figure 8(b) shows the aggregated results as scatter plot, with measures ϵ and $|S|$ segmented by part classes. All reported hard-to-solve packing problems have ϵ values below 3%, implying a high packing density of 97%–100%. These findings align with those of Clautiaux et al. (2007a), although difficult instances tend to skew towards lower ϵ values. The aforementioned authors show in their computational experiments that high area utilization is associated with increased computational difficulty and runtime. Examining the differences between classes, we observe that most hard packing problems occur for classes with small parts (classes Che, 1, and 2), while only a few hard packings are reported for classes 3 and 4. With smaller part sizes, more parts fit into a single batch. Thus, the number of parts assigned to a batch is another crucial characteristic of identifying hard packing instances. In this computational study, all hard instances have at least 12 parts; however, what consistently complicates the solution of packing instances is the interplay between the factors depicted by ϵ , $|S|$, and the composition of the actual parts (represented by the classes). To evaluate the performance of the solver GOOGLE OR-TOOLS used for the CP model, in the following section we provide a detailed comparison of alternative solution techniques for the 2D-OPR.

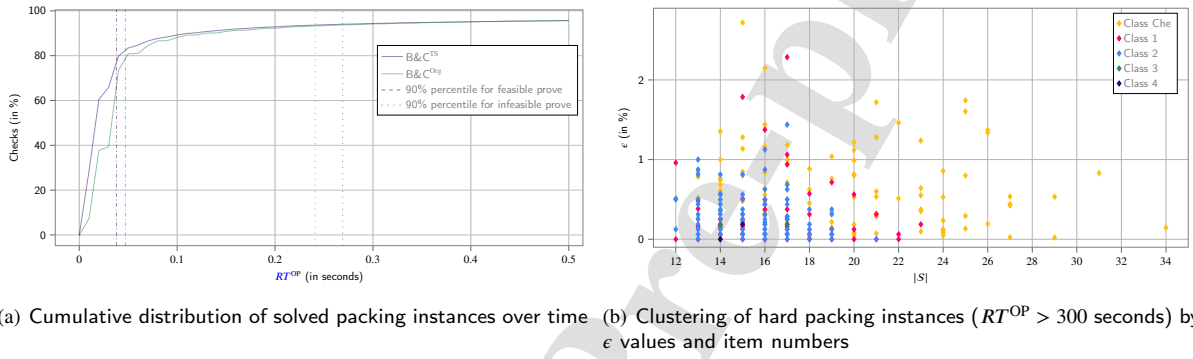


Figure 8: Detailed evaluations of using constraint programming to solve the 2D-OPR

5.3.3. Comparison of exact packing approaches

The effectiveness of the proposed B&C algorithm and its individual components has been extensively analyzed in the previous sections. Moreover, we have demonstrated that the chosen off-the-shelf solver GOOGLE OR-TOOLS can efficiently solve almost all the encountered packing problems. In this section, we compare different alternatives to solve the 2D-OPR; this comparison aligns with our commitment to utilizing non-tailored solution algorithms whenever possible, thus promoting the reproducibility of our approach. We consider three different alternatives for solving the 2D-OPR exactly:

- **B&C_{GO}^{TS}**: This variant is equal to the B&C^{TS} variant described before. We use GOOGLE OR-TOOLS to solve the 2D-OPR at the final step of the verification.
- **B&C_{IC}^{TS}**: This variant substitutes GOOGLE OR-TOOLS with a CP model formulated using IBM ILOG CP OPTIMIZER.
- **B&C_{DIM}^{TS}**: In variant B&C_{DIM}^{TS}, we use the B&C approach proposed in Delorme et al. (2017) to tackle the 2D-OPR.

All runs for each instance were performed with the same random number seeds to ensure comparability between the algorithm variants. This guarantees that the progress of the master problem is identical until the time limit is reached. The aggregated results are presented in Table 10 and Figure 9. Overall results are provided in Table 10, where column #Opt. reports the number of optimal seed runs with the number of problem instances with at least one optimal seed run in brackets, and G^{avg} and RT^{avg} show the average optimality gap in percent and the average computation time in seconds. Finally, #Checks represents the sum of packing checks conducted within each class. This metric is reported since the only distinguishing feature between the variants is the speed at which the packing problems are solved; all other components are constant. Examining the number of optimal runs in conjunction with the average optimality gaps reveals that B&C_{GO}^{TS} and B&C_{DIM}^{TS} significantly outperform B&C_{IC}^{TS} for classes with smaller part sizes, offering similar

Branch-and-cut for integrated planning in additive manufacturing

results with alternating advantages between the two variants. In contrast, all three variants show similar performance for classes 3 and 4 with larger parts and easier packing problems.

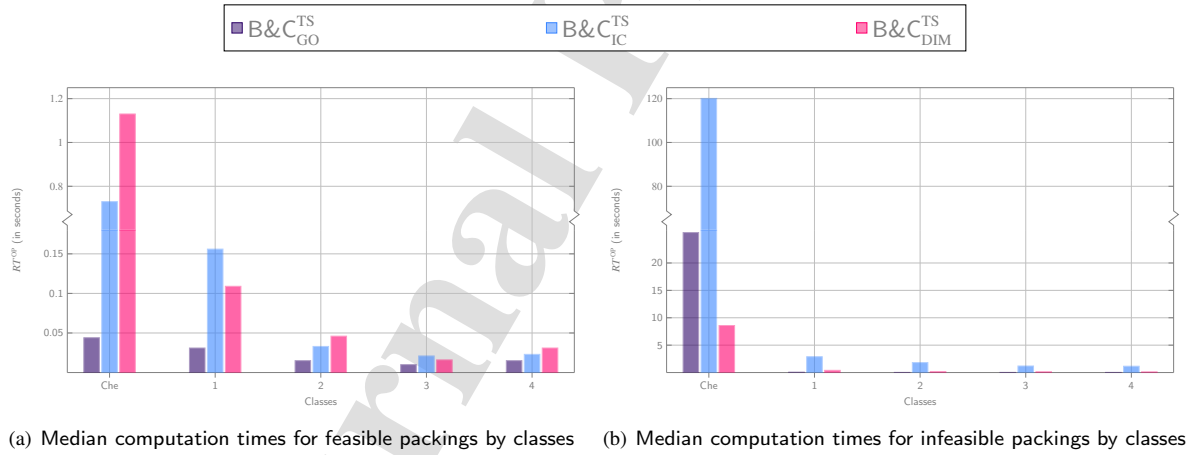
Table 10

Result summary: Impact analysis of cut lifting procedures based on selected problem instances

Class	B&C _{GO} ^{TS}				B&C _{IC} ^{TS}				B&C _{DIM} ^{TS}			
	#Opt.	G^{avg}	RT^{avg}	#Checks	#Opt.	G^{avg}	RT^{avg}	#Checks	#Opt.	G^{avg}	RT^{avg}	#Checks
Che	77 (26)	0.63	1327.93	923	73 (25)	0.85	1481.58	818	78 (27)	0.74	1266.63	983
1	81 (29)	1.43	1567.26	11401	67 (23)	2.90	1855.45	3270	81 (28)	1.10	1539.11	11882
2	56 (19)	4.89	2162.22	22091	51 (17)	6.12	2302.32	15187	55 (19)	5.30	2121.07	21671
3	51 (17)	6.75	2317.93	53064	51 (17)	6.72	2313.34	51192	51 (17)	6.98	2215.74	51192
4	48 (16)	5.07	2398.33	64582	49 (17)	5.10	2402.66	61381	49 (17)	5.23	2275.29	63980

#Opt.: Number of runs that ended with an optimal solution; number of instances with at least one optimal solution in brackets. G^{avg} : Average MIP gap of all runs within a class in percent. RT^{avg} : Average computation time of all runs within a class in seconds. #Checks: Total number of checks conducted within each class.

Figure 9 compares the median computation times for feasible and infeasible packings across the considered alternatives. Two main observations can be drawn from the presented figures. First, the implementation using GOOGLE OR-TOOLS is highly efficient at finding a feasible solution if any exist. B&C_{GO}^{TS} clearly outperforms the other two variants across all classes. Second, while B&C_{DIM}^{TS} struggles to find feasible packings, it excels in proving infeasibility for problem instances with small part sizes and a high number of parts per instance. As parts become larger across classes 1, 2, 3, and 4, B&C_{GO}^{TS} consistently reports the smallest median values in this category. Delorme et al. (2017) use a truncated version of the B&B approach proposed in (Kenmochi et al., 2009) to quickly find feasible solutions. However, its performance is strongly influenced by the implementation and programming language. Therefore, exploring a hybrid approach that combines B&C_{GO}^{TS} and B&C_{DIM}^{TS} could be a promising avenue for future research.

**Figure 9:** Comparison of computation times to solve the 2D-OPR by different alternatives

6. Conclusion

This study is motivated by the production processes in additive manufacturing systems. More precisely, its practical relevance arises from manufacturing shops that include several different additive manufacturing machines, which are used to fulfill printing-on-demand services for both inhouse and external customers. Additive manufacturing machines, or 3D printers, enable the simultaneous processing of several parts in the same batch. Therefore, we investigate the integrated problem of two-dimensional orthogonal packing with rotation and scheduling on unrelated parallel machines. To solve this problem exactly, we propose a new branch-and-cut approach that incorporates and extends several adaptations of state-of-the-art procedures and techniques from the packing literature. Through computational tests on both existing and new benchmark data, we evaluate two variants of the proposed branch-and-cut method,

demonstrating their superior performance over an integrated mixed-integer programming model from the literature. The results show that the proposed approaches yield more optimal solutions while significantly reducing computational effort. Our analysis further assesses the impact of advanced packing methods on schedules, and provide valuable insights by identifying key factors that influence packing outcomes in different problem structures. Notably, our analysis reveals that packing becomes particularly important when height-dependent processing times and setups constitute a significant portion of the total processing time. In addition, we analyze the key algorithmic components separately and show their effectiveness for the proposed approach. We observe that lower bound techniques add significant value in the solution process for instances with larger parts, while using dual feasible functions for lower bounding can further improve checks for infeasibility in the sub-problem. In summary, the considered step-wise procedure significantly decreases the need for invoking alternative feasibility checks, which are more costly in terms of computational effort.

While most packing problems can be solved with marginal computational effort, the proposed algorithms struggle to determine feasibility or infeasibility for packing problems with many small parts and a high area utilization. We explore several alternatives to solve the sub-problem using different approaches. Future research may consider hybridization between those alternatives to accelerate solving the sub-problem. Future research may also hybridize the proposed algorithm with heuristic search strategies that leverage the already proven feasible packings to create new solutions.

For practitioners, several problem extensions must be explored in order to improve real-world applicability. For example, considering minimum distances between processed parts enables ensuring a certain level of printing quality. Extending the sub-problem to a three-dimensional packing problem enables higher build space utilization for additive manufacturing technologies that allow the stacking of parts. Lastly, as we focus on the main production step of printing the parts, future research should also include post-processing steps. Post-processing may include the removal of support structures, or the cleaning and brushing of parts. Including these steps in the overall planning might influence scheduling decisions on the 3D printer and, therefore, needs to be investigated.

Apart from additive manufacturing, the considered solution methodologies may be applied to other related problems—for example, the two-dimensional bin packing problem with rotation or the two-dimensional orthogonal packing problem with rotation. Further studies should show the extent to which incorporating these methods can improve the existing benchmark results.

CRediT authorship contribution statement

Benedikt Zipfel: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Visualization, Writing - Original Draft, Writing - Review & Editing. **Felix Tamke:** Conceptualization, Methodology, Validation, Writing - Review & Editing. **Leopold Kuttner:** Conceptualization, Writing - Review & Editing.

References

- Alicastro, M., Ferone, D., Festa, P., Fugaro, S., Pastore, T., 2021. A reinforcement learning iterated local search for makespan minimization in additive manufacturing machine scheduling problems. *Computers & Operations Research* 131, 105272. doi:10.1016/j.cor.2021.105272.
- Aloui, A., Hadj-Hamou, K., 2021. A heuristic approach for a scheduling problem in additive manufacturing under technological constraints. *Computers & Industrial Engineering* 154, 107115. doi:10.1016/j.cie.2021.107115.
- Alvarez-Valdés, R., Parreño, F., Tamarit, J.M., 2009. A branch and bound algorithm for the strip packing problem. *OR Spectrum* 31, 431–459. doi:10.1007/s00291-008-0128-5.
- Amossen, R.R., Pisinger, D., 2010. Multi-dimensional bin packing problems with guillotine constraints. *Computers & Operations Research* 37, 1999–2006. doi:10.1016/j.cor.2010.01.017.
- Arbib, C., Marinelli, F., Pizzuti, A., 2021. Number of bins and maximum lateness minimization in two-dimensional bin packing. *European Journal of Operational Research* 291, 101–113. doi:https://doi.org/10.1016/j.ejor.2020.09.023.
- Arroyo, J.E.C., Leung, J.Y.T., 2017a. An effective iterated greedy algorithm for scheduling unrelated parallel batch machines with non-identical capacities and unequal ready times. *Computers & Industrial Engineering* 105, 84–100. doi:10.1016/j.cie.2016.12.038.
- Arroyo, J.E.C., Leung, J.Y.T., 2017b. Scheduling unrelated parallel batch processing machines with non-identical job sizes and unequal ready times. *Computers & Operations Research* 78, 117–128. doi:10.1016/j.cor.2016.08.015.
- Attaran, M., 2017. The rise of 3-D printing: The advantages of additive manufacturing over traditional manufacturing. *Business Horizons* 60, 677–688. doi:10.1016/j.bushor.2017.05.011.
- Bain, E.D., 2019. Polymer powder bed fusion additive manufacturing: Recent developments in materials, processes, and applications, in: *Polymer-Based Additive Manufacturing: Recent Developments*. chapter 2, pp. 7–36. doi:10.1021/bk-2019-1315.ch002.
- Baker, B.S., Schwarz, J.S., 1983. Shelf algorithms for two-dimensional packing problems. *SIAM Journal on Computing* 12, 508–525. doi:10.1137/0212033.

Branch-and-cut for integrated planning in additive manufacturing

- Baldacci, R., Boschetti, M.A., Ganovelli, M., Maniezzo, V., 2014. Algorithms for nesting with defects. *Discrete Applied Mathematics* 163, 17–33. doi:10.1016/j.dam.2012.03.026.
- Belov, G., Kartak, V., Rohling, H., Scheithauer, G., 2009. One-dimensional relaxations and lp bounds for orthogonal packing. *International Transactions in Operational Research* 16, 745–766. doi:10.1111/j.1475-3995.2009.00713.x.
- Belov, G., Kartak, V.M., Rohling, H., Scheithauer, G., 2013. Conservative scales in packing problems. *OR Spectrum* 35, 505–542. doi:10.1007/s00291-011-0277-9.
- Bennell, J.A., Lee, L.S., Potts, C.N., 2013. A genetic algorithm for two-dimensional bin packing with due dates. *International Journal of Production Economics* 145, 547–560. doi:10.1016/j.ijpe.2013.04.040.
- Berkey, J.O., Wang, P.Y., 1987. Two-dimensional finite bin-packing algorithms. *Journal of the Operational Research Society* 38, 423–429. doi:10.1057/jors.1987.70.
- Bezerra, V.M., Leao, A.A., Oliveira, J.F., Santos, M.O., 2020. Models for the two-dimensional level strip packing problem—a review and a computational evaluation. *Journal of the Operational Research Society* 71, 606–627. doi:10.1080/01605682.2019.1578914.
- Blakey-Milner, B., Gradl, P., Snedden, G., Brooks, M., Pitot, J., Lopez, E., Leary, M., Berto, F., Du Plessis, A., 2021. Metal additive manufacturing in aerospace: A review. *Materials & Design* 209, 110008. doi:10.1016/j.matdes.2021.110008.
- Boschetti, M.A., Mingozzi, A., 2003. The two-dimensional finite bin packing problem. part II: New lower and upper bounds. *Quarterly Journal of the Belgian, French and Italian Operations Research Societies* 1, 135–147. doi:10.1007/s10288-002-0006-y.
- Boschetti, M.A., Mingozzi, A., Hadjiconstantinou, E., 2002. New upper bounds for the two-dimensional orthogonal non-guillotine cutting stock problem. *IMA Journal of Management Mathematics* 13, 95–119. doi:10.1093/imaman/13.2.95.
- Boschetti, M.A., Montaletti, L., 2010. An exact algorithm for the two-dimensional strip-packing problem. *Operations Research* 58, 1774–1791. doi:10.1287/opre.1100.0833.
- Carlier, J., Clautiaux, F., Moukrim, A., 2007. New reduction procedures and lower bounds for the two-dimensional bin packing problem with fixed orientation. *Computers & Operations Research* 34, 2223–2250. doi:10.1016/j.cor.2005.08.012.
- Che, Y., Hu, K., Zhang, Z., Lim, A., 2021. Machine scheduling with orientation selection and two-dimensional packing for additive manufacturing. *Computers & Operations Research* 130, 105245. doi:10.1016/j.cor.2021.105245.
- Chergui, A., Hadj-Hamou, K., Vignat, F., 2018. Production scheduling and nesting in additive manufacturing. *Computers & Industrial Engineering* 126, 292–301. doi:10.1016/j.cie.2018.09.048.
- Chung, F.R.K., Garey, M.R., Johnson, D.S., 1982. On packing two-dimensional bins. *SIAM Journal on Algebraic Discrete Methods* 3, 66–76. doi:10.1137/0603007.
- Clautiaux, F., Carlier, J., Moukrim, A., 2007a. A new exact method for the two-dimensional orthogonal packing problem. *European Journal of Operational Research* 183, 1196–1211. doi:10.1016/j.ejor.2005.12.048.
- Clautiaux, F., Jouglet, A., Carlier, J., Moukrim, A., 2008. A new constraint programming approach for the orthogonal packing problem. *Computers & Operations Research* 35, 944–959. doi:10.1016/j.cor.2006.05.012.
- Clautiaux, F., Jouglet, A., El Hayek, J., 2007b. A new lower bound for the non-oriented two-dimensional bin-packing problem. *Operations Research Letters* 35, 365–373. doi:10.1016/j.orl.2006.07.001.
- Codato, G., Fischetti, M., 2006. Combinatorial benders' cuts for mixed-integer linear programming. *Operations Research* 54, 756–766. doi:10.1287/opre.1060.0286.
- Côté, J.F., Dell'Amico, M., Iori, M., 2014. Combinatorial benders' cuts for the strip packing problem. *Operations Research* 62, 643–661. doi:10.1287/opre.2013.1248.
- Côté, J.F., Haouari, M., Iori, M., 2021. Combinatorial benders decomposition for the two-dimensional bin packing problem. *INFORMS Journal on Computing* 33, 963–978. doi:10.1287/ijoc.2020.1014.
- Côté, J.F., Iori, M., 2018. The meet-in-the-middle principle for cutting and packing problems. *INFORMS Journal on Computing* 30, 646–661. doi:10.1287/ijoc.2018.0806.
- Dell'Amico, M., Martello, S., Vigo, D., 2002. A lower bound for the non-oriented two-dimensional bin packing problem. *Discrete Applied Mathematics* 118, 13–24. doi:10.1016/S0166-218X(01)00253-0.
- Delorme, M., Iori, M., Martello, S., 2017. Logic based benders' decomposition for orthogonal stock cutting problems. *Computers & Operations Research* 78, 290–298. doi:10.1016/j.cor.2016.09.009.
- Emde, S., Polten, L., Gendreau, M., 2020. Logic-based benders decomposition for scheduling a batching machine. *Computers & Operations Research* 113, 104777. doi:https://doi.org/10.1016/j.cor.2019.104777.
- Fekete, S.P., Schepers, J., 2004. A general framework for bounds for higher-dimensional orthogonal packing problems. *Mathematical Methods of Operations Research* 60, 311–329. doi:10.1007/s001860400376.
- Fekete, S.P., Schepers, J., Van der Veen, J.C., 2007. An exact algorithm for higher-dimensional orthogonal packing. *Operations Research* 55, 569–587. doi:10.1287/opre.1060.0369.
- Fina, F., Gaisford, S., Basit, A.W., 2018. Powder bed fusion: The working process, current applications and opportunities, in: Basit, A.W., Gaisford, S. (Eds.), *3D Printing of Pharmaceuticals*. Springer, Cham, pp. 81–105. doi:10.1007/978-3-319-90755-0_5.
- Fleszar, K., 2016. An exact algorithm for the two-dimensional stage-unrestricted guillotine cutting/packing decision problem. *INFORMS Journal on Computing* 28, 703–720. doi:10.1287/ijoc.2016.0708.
- Fowler, J.W., Mönch, L., 2022. A survey of scheduling with parallel batch (p-batch) processing. *European Journal of Operational Research* 298, 1–24. doi:10.1016/j.ejor.2021.06.012.
- Gardeyn, J., Wauters, T., 2022. A goal-driven ruin and recreate heuristic for the 2d variable-sized bin packing problem with guillotine constraints. *European Journal of Operational Research* 301, 432–444. doi:https://doi.org/10.1016/j.ejor.2021.11.031.
- GE Aerospace, 2018. Manufacturing Milestone: 30,000 Additive Fuel Nozzles. <https://www.geaerospace.com/news/articles/manufacturing/manufacturing-milestone-30000-additive-fuel-nozzles>. Accessed: 2024-10-21.

Branch-and-cut for integrated planning in additive manufacturing

- Gibson, I., Rosen, D., Stucker, B., Khorasani, M., 2021. Additive manufacturing technologies. 3 ed., Springer Cham. doi:10.1007/978-3-030-56127-7.
- Gilmore, P.C., Gomory, R.E., 1961. A linear programming approach to the cutting-stock problem. *Operations Research* 9, 849–859. doi:10.1287/opre.9.6.849.
- Guo, N., Leu, M.C., 2013. Additive manufacturing: technology, applications and research needs. *Frontiers of Mechanical Engineering* 8, 215–243. doi:10.1007/s11465-013-0248-8.
- Herz, J., 1972. Recursive computational procedure for two-dimensional stock cutting. *IBM Journal of Research and Development* 16, 462–469. doi:10.1147/rd.165.0462.
- Hooker, J.N., Ottosson, G., 2003. Logic-based benders decomposition. *Mathematical Programming* 96, 33–60. doi:10.1007/s10107-003-0375-9.
- Hu, K., Che, Y., Zhang, Z., 2022. Scheduling unrelated additive manufacturing machines with practical constraints. *Computers & Operations Research* 144, 105847.
- Iori, M., de Lima, V.L., Martello, S., Miyazawa, F.K., Monaci, M., 2021. Exact solution techniques for two-dimensional cutting and packing. *European Journal of Operational Research* 289, 399–415. doi:10.1016/j.ejor.2020.06.050.
- Johnson, D.S., 1973. Near-optimal bin packing algorithms. Ph.D. thesis. Massachusetts Institute of Technology.
- Kang, H.S., Noh, S.D., Son, J.Y., Kim, H., Park, J.H., Lee, J.Y., 2018. The FaaS system using additive manufacturing for personalized production. *Rapid Prototyping Journal* 24, 1486–1499. doi:10.1108/RPJ-11-2016-0195.
- Kapadia, M.S., Starly, B., Thomas, A., Uzsoy, R., Warsing, D., 2019. Impact of scheduling policies on the performance of an additive manufacturing production system. *Procedia Manufacturing* 39, 447–456. doi:10.1016/j.promfg.2020.01.388.
- Kenmochi, M., Imamichi, T., Nonobe, K., Yagiura, M., Nagamochi, H., 2009. Exact algorithms for the two-dimensional strip packing problem with and without rotations. *European Journal of Operational Research* 198, 73–83. doi:10.1016/j.ejor.2008.08.020.
- Kucukkoc, I., 2019. MILP models to minimise makespan in additive manufacturing machine scheduling problems. *Computers & Operations Research* 105, 58–67. doi:10.1016/j.cor.2019.01.006.
- Lee, S.J., Kim, B.S., 2023. Two-stage meta-heuristic for part-packing and build-scheduling problem in parallel additive manufacturing. *Applied Soft Computing*, 110132doi:10.1016/j.asoc.2023.110132.
- Li, Q., Kucukkoc, I., Zhang, D.Z., 2017. Production planning in additive manufacturing and 3D printing. *Computers & Operations Research* 83, 157–172. doi:10.1016/j.cor.2017.01.013.
- Li, X., Huang, Y., Tan, Q., Chen, H., 2013. Scheduling unrelated parallel batch processing machines with non-identical job sizes. *Computers & Operations Research* 40, 2983–2990. doi:10.1016/j.cor.2013.06.016.
- Liu, L., Wu, Z., Yu, Y., 2023. A branch-and-price algorithm to perform single-machine scheduling for additive manufacturing. *Journal of Management Science and Engineering* 8, 273–286. doi:10.1016/j.jmse.2022.10.001.
- Liu, Y., Chu, C., Wang, K., 2011. A dynamic programming-based heuristic for the variable sized two-dimensional bin packing problem. *International Journal of Production Research* 49, 3815–3831. doi:10.1080/00207543.2010.501549.
- Lodi, A., Tramontani, A., 2014. Performance Variability in Mixed-Integer Programming. *INFORMS TutORials in Operations Research*, 1–12. doi:10.1287/educ.2013.0112.
- Manco, P., Macchiaroli, R., Maresca, P., Fera, M., 2019. The Additive Manufacturing Operations Management Maturity: a Closed or an Open Issue? *Procedia Manufacturing* 41, 98–105. doi:10.1016/j.promfg.2019.07.034.
- Mao, Z., Fu, E., Huang, D., Fang, K., Chen, L., 2024. Combinatorial benders decomposition for single machine scheduling in additive manufacturing with two-dimensional packing constraints. *European Journal of Operational Research* 317, 890–905. doi:https://doi.org/10.1016/j.ejor.2024.05.001.
- Martello, S., Monaci, M., Vigo, D., 2003. An exact approach to the strip-packing problem. *INFORMS Journal on Computing* 15, 310–319. doi:10.1287/ijoc.15.3.310.16082.
- Martello, S., Pisinger, D., Vigo, D., Boef, E.D., Korst, J., 2007. Algorithm 864: General and robot-packable variants of the three-dimensional bin packing problem. *ACM Transactions on Mathematical Software (TOMS)* 33, 7–es. doi:10.1145/1206040.1206047.
- Martello, S., Vigo, D., 1998. Exact solution of the two-dimensional finite bin packing problem. *Management Science* 44, 388–399. doi:10.1287/mnsc.44.3.388.
- Muter, I., 2020. Exact algorithms to minimize makespan on single and parallel batch processing machines. *European Journal of Operational Research* 285, 470–483. doi:https://doi.org/10.1016/j.ejor.2020.01.065.
- Nascimento, P., Silva, C., Mueller, S., Moniz, S., 2023. Nesting and Scheduling for Additive Manufacturing: An Approach Considering Order Due Dates, in: Almeida, J.P., Geraldes, C.S., Lopes, I.C., Moniz, S., Oliveira, J.F., Pinto, A.A. (Eds.), *Operational Research*, Springer International Publishing, Cham. pp. 117–128. doi:10.1007/978-3-031-20788-4_8.
- Nascimento, P.J., Silva, C., Antunes, C.H., Moniz, S., 2024. Optimal decomposition approach for solving large nesting and scheduling problems of additive manufacturing systems. *European Journal of Operational Research* doi:10.1016/j.ejor.2024.03.004.
- Oh, Y., Witherell, P., Lu, Y., Sprock, T., 2020. Nesting and Scheduling Problems for Additive Manufacturing: A Taxonomy and Review. *Additive Manufacturing* 36, 101492. doi:10.1016/j.addma.2020.101492.
- Pinto, M., Silva, C., Thürer, M., Moniz, S., 2024. Nesting and scheduling optimization of additive manufacturing systems: Mapping the territory. *Computers & Operations Research*, 106592doi:10.1016/j.cor.2024.106592.
- Pisinger, D., Sigurd, M., 2005. The two-dimensional bin packing problem with variable bin sizes and costs. *Discrete Optimization* 2, 154–167. doi:10.1016/j.disopt.2005.01.002.
- Pisinger, D., Sigurd, M., 2007. Using decomposition techniques and constraint programming for solving the two-dimensional bin-packing problem. *INFORMS Journal on Computing* 19, 36–51. doi:10.1287/ijoc.1060.0181.
- Polyakovskiy, S., M'Hallah, R., 2018. A hybrid feasibility constraints-guided search to the two-dimensional bin packing problem with due dates. *European Journal of Operational Research* 266, 819–839. doi:10.1016/j.ejor.2017.10.046.

Branch-and-cut for integrated planning in additive manufacturing

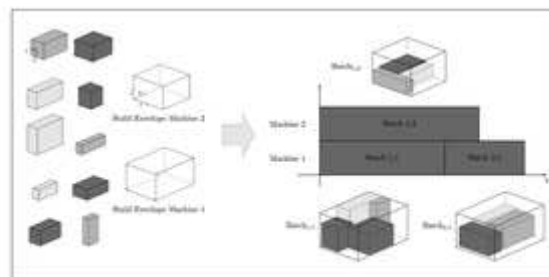
- Polyakovskiy, S., M'Hallah, R., 2021. Just-in-time two-dimensional bin packing. *Omega* 102, 102311. doi:10.1016/j.omega.2020.102311.
- Rohaninejad, M., Hanzálek, Z., Tavakkoli-Moghaddam, R., 2021. Scheduling of parallel 3d-printing machines with incompatible job families: A matheuristic algorithm, in: IFIP International Conference on Advances in Production Management Systems, Springer. pp. 51–61. doi:10.1007/978-3-030-85874-2_6.
- SLM Solutions AG, 2023. Product Information SLM 500. https://www.slm-solutions.com/fileadmin/Content/Machines/SLM_R_500_Web.pdf. Accessed: 2023-02-01.
- Tavakkoli-Moghaddam, R., Shirazian, S., Vahedi-Nouri, B., 2020. A Bi-objective Scheduling Model for Additive Manufacturing with Multiple Materials and Sequence-Dependent Setup Time, in: Lalic, B., Majstorovic, V., Marjanovic, U., von Cieminski, G., Romero, D. (Eds.), *Advances in Production Management Systems. Towards Smart and Digital Manufacturing*, Springer International Publishing, Cham. pp. 451–459. doi:10.1007/978-3-030-57997-5_52.
- Thompson, M.K., Moroni, G., Vaneker, T., Fadel, G., Campbell, R.I., Gibson, I., Bernard, A., Schulz, J., Graf, P., Ahuja, B., Martina, F., 2016. Design for additive manufacturing: Trends, opportunities, considerations, and constraints. *CIRP Annals* 65, 737–760. doi:10.1016/j.cirp.2016.05.004.
- Uzsoy, R., 1994. Scheduling a single batch processing machine with non-identical job sizes. *International Journal of Production Research* 32, 1615–1635. doi:10.1080/00207549408957026.
- Wei, L., Oon, W.C., Zhu, W., Lim, A., 2013. A goal-driven approach to the 2d bin packing and variable-sized bin packing problems. *European Journal of Operational Research* 224, 110–121. doi:https://doi.org/10.1016/j.ejor.2012.08.005.
- Yu, C., Matta, A., Semeraro, Q., Lin, J., 2022. Mathematical models for minimizing total tardiness on parallel additive manufacturing machines. *IFAC-PapersOnLine* 55, 1521–1526. doi:10.1016/j.ifacol.2022.09.606.
- Zhang, J., Yao, X., Li, Y., 2020. Improved evolutionary algorithm for parallel batch processing machine scheduling in additive manufacturing. *International Journal of Production Research* 58, 2263–2282. doi:10.1080/00207543.2019.1617447.
- Zipfel, B., 2024. Instances and test results for integrated planning for additive manufacturing. URL: <https://data.mendeley.com/preview/k4vvbvf5kb?a=6b4d984e-f13c-4eee-a87a-ba164838aa6f>.
- Zipfel, B., M'Hallah, R., Buscher, U., 2024. Scheduling for additive manufacturing with two-dimensional packing and incompatible items. *Omega* 129, 103139. doi:https://doi.org/10.1016/j.omega.2024.103139.
- Zipfel, B., Neufeld, J.S., Buscher, U., 2021. Customer order scheduling in an additive manufacturing environment, in: Dolgui, A., Bernard, A., Lemoine, D., von Cieminski, G., Romero, D. (Eds.), *Advances in Production Management Systems. Artificial Intelligence for Sustainable and Resilient Production Systems*, Springer International Publishing, Cham. pp. 101–109. doi:10.1007/978-3-030-85910-7_11.

A new branch-and-cut approach for integrated planning in additive manufacturing

Scope

We consider planning for additive manufacturing systems. Parts are produced by the available 3D printers, which are able to process several parts simultaneously within a single production step.

The integrated planning problem combines two combinatorial problems: Two-dimensional parts need to be packed in batches, while the resulting batches are sequenced on the available machines. The overall objective is to minimize the makespan.



Findings

The algorithms B&C^{Org} and B&C^{TS} show superior performance compared to an existing exact approach:

The elaborated packing approximations and solution techniques enable an efficient packing of batches.

Efficient packing strategies are especially necessary for problem configurations where parts are small compared to the production area of the machine.

Objective values are improved by up to 97.6%.

Masterproblem

Sequencing of batches



Sub-problem

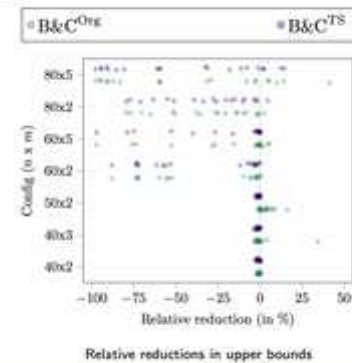
Checking for feasible batch assignments of solution candidates

Methodology

We propose a new exact algorithm (B&C) for the considered planning problem that is based on logic-based Benders decomposition.

To solve the sub-problem efficiently, we extend state-of-the-art techniques to solve the orthogonal packing problem with rotation including:

- Lower bounding techniques
- Dual feasible functions
- Orthogonal relaxation
- Placement point reduction



Highlights

A new branch-and-cut approach for integrated planning in additive manufacturing

Benedikt Zipfel, Felix Tamke, Leopold Kuttner

- We present a new exact algorithm for a scheduling problem in additive manufacturing.
- We extend state-of-the-art techniques to solve the orthogonal packing with rotation.
- The branch-and-cut algorithm is vastly superior to an existing integrated model.
- We propose new benchmark instances for the considered planning problem.